Università degli Studi di Padova

Dipartimento di Ingegneria dell'Informazione

School of Engineering - Department of Information Engineering
Master Degree in Computer Engineering

# Exploring how k-mer based compression speeds up many bioinformatics applications

*Candidate:*
Orsolon Ludovico

*Supervisor:*
Comin Matteo

Academic Year 2025-2026
Graduation date 02/04/2026

# Abstract (ENG)

This thesis focuses on USTAR [1], a k-mer based tool designed to compress large k-mer sets through an approach based on De Bruijn Graphs representations. By exploiting the structural properties of these graphs, USTAR generates compact representations of k-mer collections that are substantially smaller than the original datasets. The main objective of this work is to assess whether USTAR-based compression can be effectively integrated into bioinformatics pipelines that rely on k-mer based tools. In particular, this thesis investigates to what extent compressed datasets can be used as input for existing tools without compromising their correctness, while potentially improving computational performance and reducing storage requirements. To achieve this goal, several widely used k-mer based tools are evaluated across multiple datasets. For each tool, execution time, output characteristics, and storage usage are compared when operating on original datasets and on datasets compressed using USTAR. This comparison allows for a quantitative evaluation of the benefits introduced by USTAR in terms of computational efficiency and disk space utilization. Beyond the potential advantages, this thesis also aims to identify limitations and drawbacks introduced by dataset compression. Particular attention is given to understanding how compression affects tool accuracy, result consistency, and overall pipeline behavior. The ultimate objective is to determine under which conditions, and for which classes of applications, USTAR-based compression represents a practical and beneficial choice in real-world bioinformatics workflows.

Many of the scripts used in this thesis, along with their detailed explanations, are available in the dedicated GitHub page at github.com/OrsolonLudovico/MasterThesis.

# Abstract (IT)

Questa tesi si concentra su USTAR [1], uno strumento progettato per comprimere set di k-mer attraverso un approccio basato sui grafi di De Bruijn. Sfruttando le proprietà strutturali di tali grafi, USTAR è in grado di generare rappresentazioni compatte di questi set, di dimensioni significativamente inferiori rispetto ai dataset originali. L'obiettivo principale di questa tesi è valutare se la compressione basata su USTAR possa essere integrata all'interno di pipeline bioinformatiche che fanno uso di strumenti basati su k-mer. In particolare, la tesi analizza in che misura i dataset compressi possano essere utilizzati come input per strumenti esistenti, senza comprometterne la correttezza e migliorando al contempo prestazioni computazionali assieme a requisiti di archiviazione. Per raggiungere questo obiettivo, vengono valutati diversi strumenti basati su k-mer, applicandoli a molteplici dataset. Per ciascuno strumento vengono confrontati il tempo di esecuzione, le caratteristiche dell'output e l'uso dello spazio di archiviazione. Tale confronto consente una valutazione quantitativa dei benefici introdotti dalla compressione di USTAR. Oltre ai potenziali vantaggi, questa tesi mira anche a identificare possibili limitazioni e svantaggi che potrebbero essere introdotti da questa compressione. È stata impiegata particolare attenzione all'analisi dell'impatto della compressione sull'accuratezza degli strumenti, sulla coerenza dei risultati e sul comportamento complessivo delle pipeline. L'obiettivo finale è determinare in quali condizioni, e per quali classi di applicazioni, la compressione basata su USTAR rappresenti una scelta pratica e vantaggiosa nei reali flussi di lavoro bioinformatici.

Molti degli script utilizzati in questa tesi, insieme alle relative spiegazioni dettagliate, sono disponibili nella pagina GitHub dedicata all'indirizzo github.com/OrsolonLudovico/MasterThesis.

**1. Introduction** ..... **1**
1.1. Background ..... 1
1.1.1. k-mers ..... 1
1.1.2. Type of data used ..... 2
1.2. Thesis objective ..... 2
**2. K-mer based applications** ..... **4**
2.1. Fulgor ..... 4
2.2. GGCAT ..... 5
2.3. Mash ..... 6
2.4. REINDEER2 ..... 6
**3. Methods** ..... **8**
3.1. USTAR ..... 8
3.2. Logan ..... 8
3.2.1. Ustar's modifications to accommodate Logan ..... 9
3.3. Work pipeline ..... 9
3.3.1. DEI cluster ..... 9
3.3.2. Slurm ..... 10
3.3.3. Singularity ..... 10
**4. Datasets** ..... **11**
4.1. Human Genome (Homo sapiens) ..... 12
4.2. Human Gut bacteria ..... 12
4.3. Salmonella enterica ..... 12
4.4. Escherichia Coli ..... 13
4.5. Drosophila melanogaster ..... 13
4.6. Considerations about the compression ..... 14
**5. Initial results** ..... **15**
5.1. Assembly datasets ..... 15
5.2. Reads datasets ..... 16
5.3. Compression performances considerations ..... 17
**6. Final results** ..... **18**
6.1. Analysis complications ..... 18
6.2. Analysis clarifications ..... 18
6.3. REINDEER2 ..... 18
6.3.1. Reading the results tables ..... 19
6.3.2. Human Genome REINDEER2 results ..... 19
6.3.3. Human Gut Bacteria REINDEER2 results ..... 20
6.3.4. Salmonella Enterica REINDEER2 results ..... 20
6.3.5. Escherichia Coli REINDEER2 results ..... 20
6.3.6. Drosophila Melanogaster REINDEER2 results ..... 21
6.3.7. REINDEER2 results analysis ..... 21
6.3.7.1. REINDEER2's statistics ..... 22
6.4. Fulgor ..... 24
6.4.1. Human Genome Fulgor results ..... 24
6.4.2. Human Gut Bacteria Fulgor results ..... 24
6.4.3. Salmonella Enterica Fulgor results ..... 25

6.4.4. Escherichia Coli Fulgor results....25
6.4.5. Drosophila Melanogaster Fulgor results....26
6.4.6. Fulgor results analysis....26
6.4.6.1. Fulgor statistics....27
6.5. GGCAT....29
6.5.1. GGCAT's query performances and analysis clarifications....29
6.5.2. Human Genome GGCAT results....29
6.5.3. Human Gut Bacteria GGCAT results....29
6.5.4. Salmonella Enterica GGCAT results....30
6.5.5. Escherichia Coli GGCAT results....30
6.5.6. Drosophila Melanogaster GGCAT results....30
6.5.7. GGCAT results analysis....30
6.5.7.1. GGCAT's statistics....31
6.6. Mash....33
6.6.1. Mash analysis results explanation....33
6.6.2. Human Genome Mash results....33
6.6.3. Human Gut Bacteria Mash results....34
6.6.4. Salmonella Enterica Mash results....34
6.6.5. Escherichia Coli Mash results....34
6.6.6. Drosophila Melanogaster Mash results....35
6.6.7. Mash results analysis....35
6.6.7.1. Mash's statistics....36
6.7. Results considerations....38
**7. Conclusions and future works....39**
**8. Acknowledgments....40**
**9. Bibliography....40**
**10. Appendix....42**
10.1. Accessions lists....42
10.1.1. Human Genome....42
10.1.2. Human Gut bacteria....42
10.1.3. Salmonella enterica....42
10.1.4. Escherichia Coli....42
10.1.5. Drosophila melanogaster....42
10.2. Parameters employed....43
10.3. Data used to draw the images and graphs....43
10.3.1. Initial results....43
10.3.2. Final results....43
10.3.2.1. REINDEER2....43
10.3.2.2. Fulgor....44
10.3.2.3. GGCAT....44
10.3.2.4. Mash....45

# 1. Introduction

## 1.1. Background

*Deoxyribonucleic acid* (DNA) is composed of four repeating units known as *nucleotides*, those are: *adenine* (A), *thymine* (T), *cytosine* (C) and *guanine* (G). Nucleotides are arranged on two complementary and antiparallel strands, in which adenine pairs with thymine and cytosine with guanine through hydrogen-bond interactions, forming a double-helical structure. The ordinal sequence of the nucleotides within the DNA molecule encodes all the genetic characteristics of the individual from which the DNA originates. This nucleotide sequence determines the structure of genes, regulatory elements, and other functional genomic regions, ultimately governing the synthesis of proteins and a wide variety of functional molecules. Variations in the DNA sequence represent biological diversity, influence phenotypic traits, and can be responsible for disease predisposition. For these reasons, DNA represents the fundamental molecular basis of heredity and life itself. The rapid development of high-throughput sequencing technologies has made it possible to generate massive amounts of DNA sequence data at unprecedented speed and scale. This technological progress has brought significant advancements in genomics, medicine, and evolutionary biology but it has also introduced significant computational challenges related to the storage, processing, and analysis of large-scale genomic data. Addressing these challenges requires efficient algorithmic and data-structural approaches, a need that has driven the growth of bioinformatics as a central discipline at the intersection of biology and computer science. In this context, sequence analysis techniques based on short substrings of DNA, known as *k-mers* (Section 1.1.1), have become a fundamental building block for many bioinformatics applications, including genome assembly, indexing, and comparative genomics. Efficient representations of such data are therefore crucial for enabling scalable and accurate analysis of modern genomic datasets.

### 1.1.1. k-mers

K-mers are substrings of fixed length k. Dividing a string into k-mers means to extract from that string every substring with length k. K-mers have a central role in many bioinformatics applications even outside of assembly; some of such applications are the analysis of genome structure ( Genomescope2.0 [10] is an example), genome assembly validation (KAT [11] is an example), genome comparison (Mash [8] is an example) and k-mer index/search (REINDEER2 [6] and Fulgor [7] are examples) but there are many more. K-mers provide a compact, local, and uniform representation of biological sequences. Unlike whole genomes or long reads, which are large, heterogeneous, and difficult to compare, k-mers break sequences into fixed-size units that are easier to store, index, and process efficiently. Another key advantage of k-mers is that they avoid the need for global alignment, which is computationally expensive and often impractical at large scale. Many bioinformatics problems can be reduced to operations on k-mer sets, problems such as counting, matching, or comparing for example, making it possible to design algorithms with predictable performance and strong scalability properties. For these reasons, k-mers have become a foundational abstraction in modern bioinformatics. All the tools employed in this thesis are based on k-mers and leverage this representation to perform different tasks, ranging from indexing and querying to similarity estimation.

### 1.1.2. Type of data used

Sequencing a genome means finding the order of nucleotides that compose the genetic sequence, since the two strands are complementary it is sufficient to sequence a single strand, as the sequence of the opposite strand can be inferred by producing the first strand's complementary.
To store this sequencing information it's common to use the *FASTA* file format, which is a text-based format for storage and representation of biological sequences. A FASTA file consists of one or more sequences: each one has a *header* line that starts with the symbol ">" followed by a unique sequence identifier and one or more lines containing the sequence itself, which is represented by using a single letter (A, C, T, G) for each nucleotide.
The raw output of the sequencing process often consists in a large amount of short genomic sequences referred as *reads*, which commonly present a large amount of overlapping between each other. This means that some areas of the original genome will be represented more than one time among the totality of the reads produced by the sequencing process.
It is possible to assemble these reads using assembly algorithms, constructing long continuous sequences called *assemblies*. Compared to the reads, assemblies have much less redundancy but they are complex to produce due to sequencing errors, repetitive regions and uneven coverage.
Many assembling approaches are based on *De Bruijun graphs* (DBGs). In this type of approach sequencing reads are decomposed into fixed length substrings called *k-mers* (Section 1.1.1); each k-mer represents a vertex in the DBG, while directed edges connect k-mers that have an overlap of k - 1 nucleotides; the genomic sequence is then reconstructed by finding paths in the DBG. A *unitig* is a path inside the DBG in which each internal node has in-degree and out-degree equal to one, this means that there is no ambiguity on how the path should be expanded in each direction. This lack of ambiguity allows unitigs to be assembled easily making them an intermediate representation between reads and assembled genomes.
Reads, assembled genomes and unitigs can all be stored in the FASTA format, this format is assumed for the totality of this thesis.

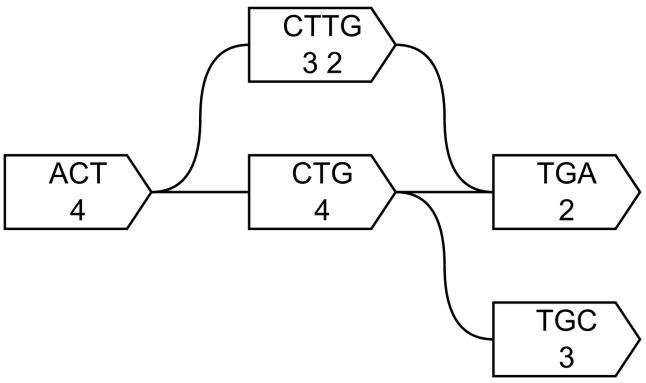


*Figure 1. An example of a compacted de Bruijn graph. Nodes are labeled with k-mers and their counts. A path on the DBG is a genomic sequence. Image taken from [1]*

## 1.2. Thesis objective

The primary objective of this thesis is to investigate the effectiveness of USTAR [1] (a k-mer based compression tool that exploits de Bruijn graph representations) in improving the efficiency of k-mer based bioinformatics applications. This work aims to evaluate whether compressing k-mer datasets using USTAR can lead to significant benefits in terms of computational performance and storage requirements, without hindering the correctness of the tools' results. For this objective, several k-mer based tools, developed to perform different tasks, will be utilized with multiple datasets in

two distinct configurations: using the original, uncompressed k-mer sets (which could be read, assemblies or unitigs) and using datasets compressed by USTAR. By rigorously comparing execution time, resource consumption, and outputs across these two configurations, this thesis aims to quantify the impact of USTAR-based compression on practical bioinformatics pipelines. In particular, the work compares the two results obtained with the two configurations to understand the differences between the two results, with and without the compression. In addition to identifying potential advantages, this work also analyzes possible limitations and/or trade-offs that could be introduced by the k-mer compression, such as a possible lowering in accuracy. Through this comprehensive evaluation, this thesis intends to clarify under which conditions, and for which applications, USTAR (and so k-mer compression) represents a beneficial initial step to obtain better, faster and/or smaller results.

# 2. K-mer based applications

In this section are reported all the bioinformatics applications used to evaluate USTAR's performance (as mentioned in Section 1.2) in this thesis, along with why they have been chosen, how they work and what parameters that have been used with them. Note that, for all the tools listed below, the k-mer size is always set to 31, which is one of the most common values, employed by a vast array of bioinformatics tools. Details and explanation of scripts and implementations mentioned in this section are available at the GitHub page dedicated to this thesis [2]. The results obtained by these datasets are reported in Section 6

## 2.1. Fulgor

The problem of sequence identification is central to many bioinformatics applications, including metagenomics and pangenome analysis. For this reason a wide range of tools for indexing and querying sequence datasets have been developed. Among these, Fulgor [7] is one of the best solutions, providing a fast and compact k-mer index for large-scale matching and color queries. Fulgor is based on a colored DBG, in which each node corresponds to a k-mer in the reference collection and is annotated with a color, representing the set of sequences in which that k-mer occurs. The tool is implemented efficiently in C++, allowing for the representation of the data structure in a compact way, while simultaneously enabling efficient color retrieval. In particular, Fulgor implements a two-level layout that combines a k-mer dictionary and an inverted index storing colors information.
Although Fulgor outperforms many existing tools it inevitably presents significant slow downs when the input k-mer datasets are particularly large. In such scenarios compressing those inputs using tools such as USTAR could significantly reduce computational time by decreasing the dataset size. During index construction, Fulgor only verifies the presence or absence of k-mers within sequences: a binary operation that does not depend on k-mer abundance. This characteristic makes Fulgor particularly well suited for integration with USTAR, as the compression process should not affect indexing accuracy when k-mer counts are not considered, as is the case in this thesis.
Fulgor does not provide a built-in mechanism to filter k-mers with abundance equal to one. As a consequence, some inconsistencies have been observed when comparing indexes built using READS vs unitigs from Logan. While Logan unitigs are filtered during construction, read-based datasets are not, leading to differences in the indexed k-mer sets. A detailed discussion of these discrepancies and their impact on query results is provided in the results section dedicated to Fulgor (Section 6.4). All parameters used for index construction and querying with Fulgor in this thesis were left at their default values, with the exception of maximum memory usage and the number of threads, which were adjusted to appropriate levels and kept consistent across all experiments.

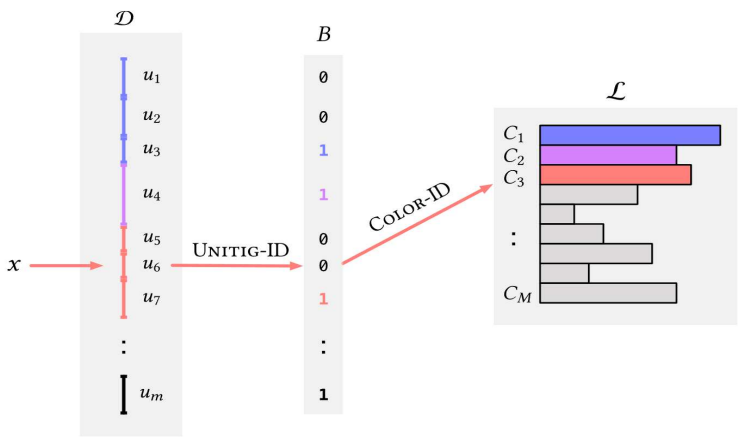


*Figure 2. A representation of the indexing strategy adopted by Fulgor. Highlighting the interplay between the k-mer dictionary $\mathcal{D}$, the bit-vector B, and the inverted index $\mathcal{L}$. The index is queried for a k-mer x, assuming that x occurs in unitig u6 and has color C3. Image taken from [7]*

## 2.2. GGCAT

GGCAT [9] is a tool designed for efficient construction of compacted, and optionally colored, DBGs starting either from raw sequencing data or by merging multiple existing cDBG in a single graph. In addition to construction, GGCAT also supports sequence queries to both colored or non-colored graphs. GGCAT is efficiently implemented in Rust, with a particular focus on parallelization, enabling effective utilization of multi-core systems and lowering the dependency on RAM's speed. This design makes GGCAT particularly suitable for large-scale datasets and high-throughput computational environments.

A distinctive feature of GGCAT is that it explicitly preserves k-mer abundance information for each node in the graph, where each node corresponds to a k-mer. This behaviour differs from other tools mentioned in this thesis, such as Fulgor, which models k-mer occurrence as a simple binary presence/absence relationship. Retaining k-mer abundance enables more flexible and accurate filtering strategies, allowing k-mers with abundance below a user-defined threshold to be removed directly during graph construction. This capability eliminates inconsistencies that could arise by selecting different abundances for the pre-filtering of the various datasets.

In this thesis, GGCAT has been employed in two different ways: firstly, as a stand alone tool for DBG construction and query while secondly, as an intermediate processing step within the REINDEER2 pipeline, as described in Section 2.4. In this work the minimum k-mer abundance threshold is always set to two. However, when operating on datasets derived from the Logan repository, which are already filtered for abundance, the threshold is set to one in order to avoid filtering two times. This configuration is applied in the stand-alone GGCAT workflow as well as in the REINDEER2 pipeline.

All other parameters utilized in this thesis were left at their default values. The only adjustment concerned the maximum number of utilizable threads, which was tuned to match the characteristics of the DEI cluster and kept constant across all experiments.

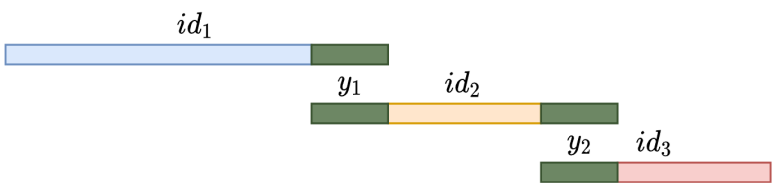


*Figure 3. The result of the intermediate unitig construction happening inside GGCAT. Each intermediate unitig that has a possible extension shares an ending with another intermediate unitig. Image taken from [9]*

## 2.3. Mash

Mash [8] is a general purpose toolkit that utilizes the MinHash technique [17], a form of locality-sensitive hashing, to reduce large sequences (or sequence sets) into compact sketch representations. Using only the sketches, which can be thousands of times smaller than the original data, the similarity of the original sequences can be rapidly estimated with bounded error. Thus, sketches consisting of just a few hundred values can be used to approximate the similarity of arbitrary large datasets. Mash is widely adopted in many bioinformatics workflows and is well suited for tasks such as clustering or real-time genome identification for assemblies of reads.
In the context of this thesis, Mash was employed with a specific focus on sketch construction and comparison. Two distinct sets of sketches are generated: one from the original, uncompressed read datasets and one from their compressed counterparts. The compressed datasets are obtained either by compressing reads directly or by operating on Logan unitigs, depending on the experimental setting, and are always produced using USTAR. Following sketch construction, Mash's "*dist*" command was used to compute pairwise distances among all sequences in both the compressed and uncompressed datasets. Ideally, the distances between corresponding sequences should be the same for both sets, not changing when computed in the compressed dataset. While the results presented later show that this condition is not met exactly, the observed differences are always limited. This analysis provides valuable insight into how USTAR-based compression affects the general sketching process.
Note that Mash, as REINDEER2 and GGCAT, supports the k-mer abundance-based filtering of the input, allowing for the discard of single abundance k-mer, probably generated by sequencing errors. All parameters of Mash were left as the default ones during this thesis.

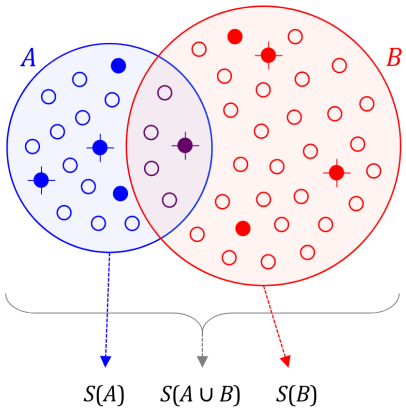


*Figure 4. Overview of part of the MinHash bottom sketch strategy for estimating the Jaccard index inside Mash. First, the sequences of two datasets are decomposed into their constituent k-mers, then each k-mer is passed through an hash function "h" to obtain a hash. The resulting hash sets are A and B, containing |A| and |B| distinct hashes each. The jaccard index is computed as J=|A∩B| / |A∪B| ≈ |S(A∪B)∩S(A)∩S(B)| / S(A∪B) . Mash's similarity is then based on Jaccard index. Image taken from [8]*

## 2.4. REINDEER2

The REINDEER2 [6] index is an highly scalable inverted index built for retrieving sequence abundance information across large-scale sequencing datasets. Unlike other existing indexing tools, like the previously mentioned Fulgor, REINDEER2 does not model k-mer indexing as a simple presence/absence problem; instead, it explicitly preserves k-mer abundance information, enabling a broader range of analyses.
REINDEER2 relies on the previously described GGCAT, it mainly employs it for k-mer extraction and count while adding on top of it a fast and robust indexing and querying logic. To achieve this it combines an inverted index with both probabilistic and exact data structures, allowing scalable

storage and fast retrieval of abundance information. Consequently, indexing with REINDEER2 requires a two-step process: it's first necessary to run GGCAT on the input datasets, and only afterwards it's possible to use the "*index*" command of REINDEER2.

As discussed in Section 6.5.7, overall, REINDEER2 introduces a modest amount of computational overhead compared to running GGCAT alone, but produces substantially more compact indexes. The results also show that REINDEER2 is significantly slower than Fulgor during index construction, while generating much smaller indexes and enabling faster query execution (Section 6.4.6). Furthermore, by utilizing GGCAT's preprocessing capabilities, REINDEER2 supports filtering of low-abundance k-mers directly from the input datasets. This feature makes it possible to selectively filter unfiltered datasets while avoiding redundant or erroneous filtering of datasets that have already been processed.

Despite these capabilities, as mentioned before, k-mer abundance information is not considered in the main analyses presented in this thesis. Although experiments incorporating abundance information were initially conducted, it was ultimately clear that simply compressing sequences with USTAR loses the abundance-related information in the context of this tool, making direct comparisons between compressed and uncompressed datasets unreliable. It should be noted however that USTAR can preserve k-mer counts and that, in principle, such information could be integrated into REINDEER2-based analyses. Exploring this direction is left for future work, as it lies beyond the scope of this work.

All parameters used in the REINDEER2 experiments were left at their default values, with the sole exception of the number of CPU cores, which was specified to match the available cluster resources and kept consistent among the various experiments. The parameters used for the GGCAT component of the REINDEER2 pipeline were set exactly as described in the GGCAT section of this chapter (Section 2.2).

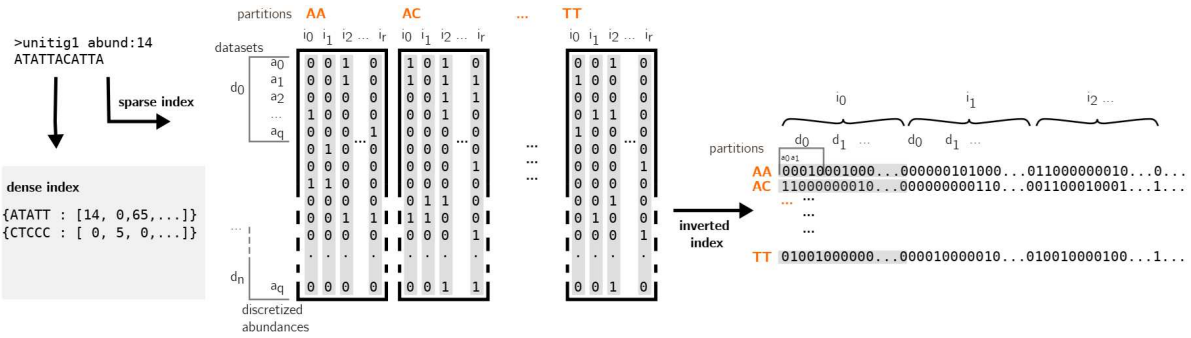


*Figure 5. Simplified view of the REINDEER2 index. Left: dense index, where k-mer occurring in all or most datasets are stored with explicit and exact abundances. Middle: abstract idea of the sparse structure, where we have one Bloom filter per dataset per abundance (row-wise Bloom filters). Right: actual inverted index of our sparse structure. Each line is a single Bloom filter containing the information of several datasets. Partitioning is shown with lexicographic minimizers of size 2. The first slice of each partition is shown in gray. Figure taken from [6]*

# 3. Methods

Details and explanation of scripts and implementations mentioned in this section are available at the GitHub page dedicated to this thesis [2]

## 3.1. USTAR

K-mer based tools often rely on specialized data structures for representing k-mer sets. Given the massive size of modern sequencing datasets, the memory footprint of such data structures can become a bottleneck when attempting to scale up to large databases. It has been shown [15] that, in the worst case, at least $log\left(4^k/n\right)$ bits are required to losslessly store a set of $n$ k-mers. However, in practical scenarios, k-mer sets typically exhibit spectrum-like properties, along with considerable redundancy. So, by exploiting these features, many data structures are able to achieve storage requirements that are significantly lower than the theoretical worst-case bound.
A practical approach to k-mer set compression consists of constructing a DBG from the input k-mer set. From that, the problem of finding the smallest k-mer set representation, becomes equivalent to the problem of finding the smallest path cover in the DBG. USTAR [1] (Unitig STitch Advanced constRuction) is a k-mer based tool designed to compress a set of k-mers and their counts based on this formulation. By exploiting the connectivity and density properties of the DBG, USTAR performs an efficient selection of paths for the construction of the path cover, resulting in a highly compact representation of the original k-mer set.
USTAR plays a central role in this thesis as all k-mer set compressions performed in this work have been carried out using this tool. Although USTAR is capable of compressing both k-mers and their abundance information, the tools selected in this thesis do not exploit k-mer counts and consequently, whenever USTAR-based compression is mentioned in this thesis, all the k-mer abundance information is discarded. It is important to note that with USTAR's compression all k-mers which are present in the input dataset are preserved during the compression, no k-mer is ever discarded. This holds true regardless of whether counts are preserved or not.
USTAR relies on BCALM2 [3] for the construction of unitigs (USTAR's input) using a low-memory footprint. When starting from sequencing reads, BCALM2 must therefore be executed prior to USTAR in order to generate the unitig representation of the underlying DBG. In cases where the input datasets already consist of unitigs, the BCALM2 step can be skipped, allowing USTAR to be executed directly and resulting in a significantly faster compression.
Throughout all the carried out experiments, the minimum k-mer abundance threshold was set to one, ensuring that no k-mers were discarded prior to compression. Additionally, USTAR was executed using its most effective heuristic configuration, specifically enabling the *-s+aa* and *-x-c* flags. Finally, increasing the minimum abundance yields faster compression but also significantly decreases the accuracy. For this reason, after some initial experiments, it has been decided to always use one as the minimum abundance

## 3.2. Logan

Logan [4] is a large-scale repository of DNA and RNA sequences, constructed in December 2023. It was generated by performing genome assembly over the entirety of the NCBI Sequence Read Archive [16] available up to that date, resulting in a comprehensive collection of assembled genomic sequences derived from a wide range of organisms and sequencing projects.
As discussed in Section 3.1, USTAR operates on unitigs and so, when the input dataset consists of raw sequencing reads, it relies on BCALM2 to convert those reads into unitigs prior to compression. However, as observed in the experiments presented in this thesis, this compression

greatly increases the execution time of USTAR. Consequently, starting from raw reads is not ideal for performance-oriented analyses. In contrast, using a resource such as Logan, which directly provides unitigs, enables substantially faster execution of USTAR and results in a more efficient and robust analysis pipeline. For these reasons, all final results presented in this thesis are obtained using unitigs downloaded directly from the Logan repository.
It is important to note that the unitigs included in Logan are filtered using a minimum k-mer abundance threshold of two. As a result, k-mers occurring only once are considered sequencing errors and are removed during the unitig construction. This filtering strategy is standard practice in many bioinformatics applications because it generally improves data reliability, however, it also implies that downstream tools should not apply additional abundance-based filtering, as many tools do by default. Applying such filtering again would result in redundant data removal, potentially biasing the results and complicating comparisons with results coming from analyses performed on read-based datasets.

### 3.2.1. Ustar's modifications to accommodate Logan

In order for USTAR to operate directly on the unitigs provided by Logan, without requiring conversion through BCALM2, the unitig files must be compatible with USTAR's expected input format. BCALM2 also outputs unitigs, just as the ones provided by Logan; however, the two differ in the way sequence headers are formatted. In particular, BCALM2 provides the abundance of each k-mer composing a unitig directly in the sequence header while Logan reports only the average k-mer abundance for each unitig. This means that the flag *-s+aa*, which only considers the average of those numbers, can work with this setup while other heuristics can not. For this reason, only that flag is used throughout this thesis. This choice should not be regarded as a limitation, as the flag corresponds to the default heuristic employed by USTAR, which is also the best performing one according to the tool's documentation.
To accommodate this discrepancy in input format, some modifications were applied to USTAR's source files. All changes, along with the modified files and a detailed explanation of the adaptations, are provided in the "/ModifyUSTAR/mods" directory of the GitHub repository associated with this thesis [2].

## 3.3. Work pipeline

For this thesis a substantial amount of computational power was required; for this reason the *Department of Information Engineering* (DEI) computer cluster, named "Blade" [12], was utilized (Section 3.3.1). The availability of significant computational resources allowed the execution of the many tools considered in this work, while maintaining the flexibility to experiment with different parameters and flags without concern about CPU, storage or memory limitations.
A GitHub repository dedicated to this thesis was created [2]: it contains several examples of the script used throughout this work, along with their detailed explanations. Scripts used to run the tools and to build Singularity containers are also provided. The repository aims to explain the details about the implementation that are not reported here.

### 3.3.1. DEI cluster

Blade [12] is a linux-based computer cluster belonging to the Information Engineering department in Padova. It is composed of multiple independent computing nodes, divided into CPU nodes, referred as "Runners", and GPU nodes, referred as "gpus". For the purpose of this thesis, only CPU nodes were used, as GPUs do not provide significant acceleration for the type of bioinformatic workloads required by the tools considered. Access to Blade is performed through

SSH connection with a text-based interface. While simple commands such as “cat” or “head” can be executed directly from the terminal, programs requiring a larger amount of computational resources have to be executed as “jobs” using the SLURM format (Section 3.3.2). In this case, users are required to specify the numbers of CPU cores, the amount of memory and the maximum computational time that each job needs. To install new programs on Blade, the use of Singularity images (Section 3.3.3) is encouraged: for this thesis all the programs utilized were installed on a Singularity image which was built locally, and executed inside the cluster through SLURM job files.

### 3.3.2. Slurm

*Simple Linux Utility for Resource Management* (SLURM) [13] is an open-source, fault-tolerant and highly scalable job scheduling system developed for Linux clusters. It is widely adopted and deployed on many of the world’s biggest supercomputers and computer clusters. Slurm provides a framework for starting, executing and monitoring works, including parallel workloads, on a set of allocated computing nodes. It also manages simultaneous access to shared resources and maintains a queue of pending work.
A SLURM file is structurally similar to a bash script: it contains shell commands, loops and variables executed sequentially. The main difference is that, at the start of the file, the user must specify the amount of computational resources required for the job such as the number of CPU cores, memory and the maximum execution time.
When a SLURM file is submitted, the corresponding job is placed in a queue and executed once the requested resources become available. The scheduling process also takes into account factors such as the number of jobs submitted by a user and total amount of requested resources, which may influence the job’s execution priority.

### 3.3.3. Singularity

Singularity [14] is a container technology that enables the creation of container images that can be mounted on top of the filesystem of any machine with Singularity installed; these containers can include software installations as well as auxiliary files, specified at creation time. Once the container image is mounted, the system can access and execute all the programs installed within the container as if they were installed locally. This behaviour is particularly well suited for the type of workload performed in this thesis; in fact all tools needed for the many performed analyses were installed on a Singularity image, which was built on a local machine and subsequently transferred to the DEI cluster. This approach eliminates the need to install software directly on the cluster, as it is sufficient to copy the container image to the server.
A singularity image can be invoked through standard bash commands, which can themself be included in a SLURM job script. As a result, when a SLURM job is executed, the programs contained within the Singularity image are seamlessly executed within the allocated computational environment, even though they were not locally installed in the server.

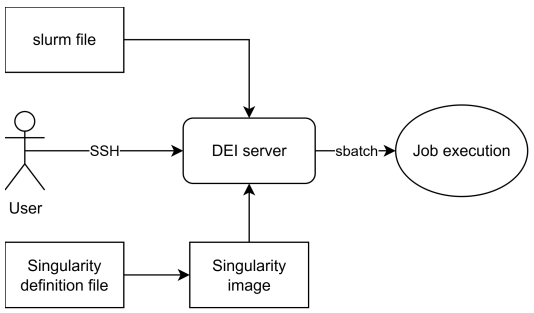


*Figure 6. Singularity usage schema on DEI server*

# 4. Datasets

This section describes the datasets used throughout this thesis. The findings reported in the “Final Results" section are based on the outputs produced by the selected tools when applied on these datasets, each one consisting of a collection of accession files selected from a specific taxa. For each selected accession, two versions of the corresponding data were obtained: the first version consists of the raw sequencing reads associated with the accession and was downloaded from the NCBI SRA [16], while the second version consists of the corresponding unitigs, which were downloaded from the Logan repository [4]. Together, the collection of read files and the collection of unitig files constitute two alternative representations of the same dataset. Starting from the unitig representation, USTAR is subsequently used to compress the unitig set, producing a third version of the dataset composed of compressed unitigs. By running the considered tools on both the uncompressed read-based datasets and the compressed unitig-based datasets, it is possible to evaluate USTAR’s performances.

Using unitigs as the input for compression is particularly advantageous, as it allows USTAR to operate directly without relying on BCALM2. As a result, the compression process is significantly faster without any loss of accuracy.

Datasets composed exclusively of reads or of assembled genomes were also initially considered, and several experiments were conducted using these representations. However, the results obtained with these approaches were not satisfactory, leading to a revision of the experimental strategy. The outcomes of these preliminary experiments are briefly reported in Section 5.

For the tables reported in this chapter, the “Gain” column quantifies the amount of storage space saved by applying USTAR’s compression on the datasets. The gain is expressed through two values: the first represents the percentage of space saved when compressing unitigs with respect to the corresponding reads-based dataset, while the second indicates the same but with respect to the uncompressed unitig dataset.

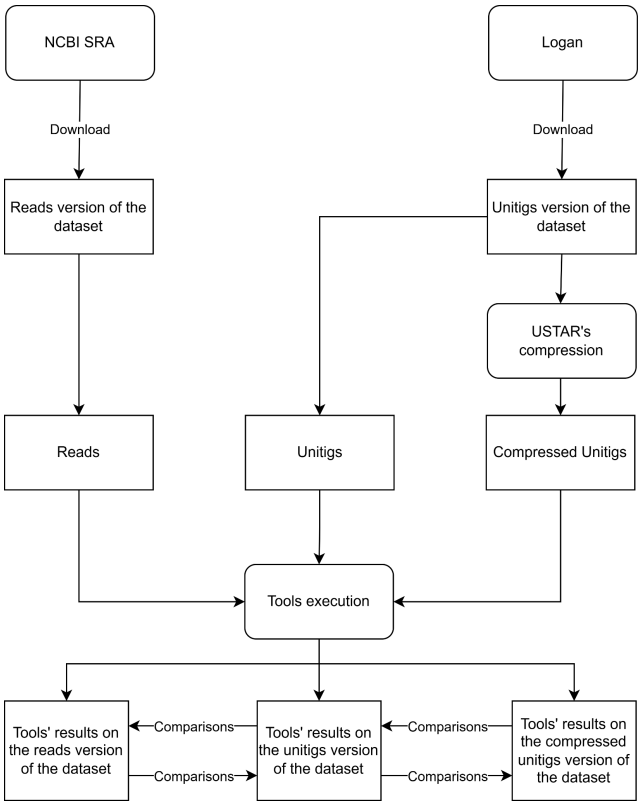


*Figure 7. The work pipeline of this thesis, the figure shows an example using reads as the starting point, the two datasets downloaded contain the same accessions*

## 4.1. Human Genome (Homo sapiens)

The Human Genome dataset comprises sequencing data from “Homo sapiens”, representing a large and highly complex eukaryotic genome. With a genome size of approximately 3.2 billion base pairs and a high degree of repetitive content, human genomic data constitute a challenging benchmark for k-mer based indexing and compression tools. In the context of this thesis, this dataset is particularly useful for evaluating the scalability of USTAR paired with the considered tools when applied to large, repetition-rich genomes, where redundancy is expected to play a major role in compression efficiency.

| | READS | Unitigs | Compressed Unitigs | Gain |
|---|---|---|---|---|
| Files | 13 | | | - |
| Size | 30.612 GB | 5.413 GB | 2.024 GB | - 93.39% / - 62.61% |
| Time | - | - | 4m 55s | - |

*Table 1. H.Genome compression rates*

## 4.2. Human Gut bacteria

The Human Gut dataset consists of sequencing data derived from the “human gut microbiome”, a complex microbial community composed of hundreds of bacterial species with highly heterogeneous abundances. Unlike single-organism genomes, metagenomic datasets are composed of genomic data belonging to a community of organisms and thus are characterized by extreme diversity, uneven coverage, and significant overlap among related species. This dataset is therefore well suited for assessing the behavior of USTAR in realistic metagenomic scenarios, where redundancy, noise, and variability are intrinsic properties of the data.

| | READS | Unitigs | Compressed Unitigs | Gain |
|---|---|---|---|---|
| Files | 39 | | | - |
| Size | 91.257 GB | 16.095 GB | 5.945 GB | - 93.49% / - 63.06% |
| Time | - | - | 14m 46s | - |

*Table 2. H.Gut Bacteria compression rates*

## 4.3. Salmonella enterica

The Salmonella enterica dataset is based on sequencing data from “Salmonella enterica”, a bacterial species of major clinical and epidemiological importance. The dataset includes multiple strains belonging to the same species, which share a high degree of genomic similarity. As a result, Salmonella enterica datasets exhibit substantial k-mer redundancy, similarly to other single-species bacterial collections. This dataset is therefore well suited for evaluating USTAR’s effectiveness in compressing highly redundant bacterial genomes and for analyzing the impact of compression on k-mer based tools when dealing with closely related genomes.

| | READS | Unitigs | Compressed Unitigs | Gain |
|---|---|---|---|---|
| Files | 38 | | | - |
| Size | 33.544 GB | 5.657 GB | 2.095 GB | - 95.75% / - 62.97% |
| Time | - | - | 4m 33s | - |

*Table 3. Salmonella enterica compression rates*

## 4.4. Escherichia Coli

The E. Coli dataset is based on sequencing data from “Escherichia coli”, one of the most extensively studied bacterial species in genomics. Due to the large number of available strains and the high level of sequence similarity among them, E. Coli datasets often exhibit substantial k-mer redundancy, similarly to the previously mentioned Salmonella genomes. This characteristic makes the dataset particularly relevant for evaluating USTAR’s ability to exploit redundancy in closely related genomes, as well as for testing the impact of compression on downstream tools.

| | READS | Unitigs | Compressed Unitigs | Gain |
|---|---|---|---|---|
| Files | 33 | | | - |
| Size | 109.033 GB | 20.492 GB | 7.372 GB | - 93.24% / - 64.02% |
| Time | - | - | 3m 39s | - |

*Table 4. E.Coli compression rates*

## 4.5. Drosophila melanogaster

The Drosophila dataset includes sequencing data from “Drosophila melanogaster”, an organism widely used in genetics and developmental biology; its genome was one of first complex genomes to be fully sequenced and thus it has a long history and relevance among bioinformatics applications. The Drosophila has an intermediate size genome, between bacterial and human genomes, combining eukaryotic complexity with a comparatively well-annotated and compact structure. This dataset provides a useful middle ground for assessing how USTAR and k-mer based tools perform on eukaryotic genomes that are less repetitive and smaller than the human genome, still being more complex than prokaryotic ones.

| | READS | Unitigs | Compressed Unitigs | Gain |
|---|---|---|---|---|
| Files | 25 | | | - |
| Size | 28.726 GB | 4.328 GB | 1.666 GB | - 94.20% / - 61.51% |
| Time | - | - | 16m 10s | - |

*Table 5. Drosophila compression rates*

## 4.6. Considerations about the compression

The observed compression rates are consistent across all datasets: the reduction in disk space with respect to the read-based datasets is consistently close to 95%, while the reduction with respect to the unitig-based datasets is always above 60%.
Reads-based datasets contain a large amount of redundant information due to sequencing coverage and overlapping reads; therefore, a substantial compression gain is to be expected when using this type of data. However, it is important to note that the unitig datasets, which are coming from Logan, are already filtered using a minimum k-mer abundance of two, as discussed previously. This filtering removes single-abundance k-mers, which are typically associated with sequencing errors, and alone accounts for a significant portion of disk space reduction, which in turn can not be considered an advantage intrinsic to USTAR.
Disregarding the filtering aspect, it is simple to observe that USTAR is also very effective at compressing the unitigs datasets; clearly, discarding k-mer count information and optimizing the unitigs structure through a DBG representation, is highly beneficial to disk space consumption, providing significant gains.

# 5. Initial results

Before settling on the experimental pipeline described in the “Methods” section, several alternative approaches were explored. This section reports some of these preliminary experiments that did not produce satisfactory results, discussing the reasons for their limited effectiveness and what can be learned from these unsuccessful attempts. Since these experiments do not represent the final outcome of this work, some details are intentionally omitted for the sake of clarity; the final results are reported with a greater level of detail, in Section 6.

## 5.1. Assembly datasets

The first datasets considered were the ones composed by genome “assemblies”. Assemblies are the results of the “assembly” process, which reconstructs a small and accurate representation of the original genome starting from sequencing reads. This process removes the redundancies present in the reads data almost entirely; redundancies that would be otherwise exploited by USTAR to achieve high compression rates. As a consequence, when applied to assembled genomes, USTAR achieves relatively low compressions while incurring in higher than expected computational costs.
Two representative examples of this behavior, obtained from different Salmonella assembly datasets, are reported in the tables in below:

| | Without USTAR | With USTAR | USTAR’s gain |
|---|---|---|---|
| Assemblies | 4589 Genomes | | - |
| Size uncompressed | 21.94 GB | 21.86 GB | - 0.36% |
| Time | (No elaboration needed) | 18h 31m | - |

*Table 6. Salmonella assembly 1*

| | Without USTAR | With USTAR | USTAR’s gain |
|---|---|---|---|
| Assemblies | 10000 Genomes | | - |
| Size uncompressed | 47.13 GB | 45.74 GB | - 2.95% |
| Time | (No elaboration needed) | 1D 15h 28m | - |

*Table 7. Salmonella assembly 2*

Although the downstream tools produced correct and high-quality results when applied to these compressed datasets, the combination of low compression gains and high computational overhead makes the use of USTAR on assembly-based datasets impractical. In this context, USTAR’s compression does not provide a meaningful advantage in reducing storage requirements; instead, it introduces additional processing costs that outweigh the benefits. For this reason, assembly-based datasets were excluded from the final experimental pipeline.

## 5.2. Reads datasets

Since datasets based on genome assemblies have limited redundancy, the most straightforward modification to the experimental pipeline was to apply USTAR directly to read-based datasets, where substantially higher compression rates are expected due to the inherent redundancy of sequencing reads.
The compression statistics obtained for three representative read-based datasets are reported in the tables below:

| | Without USTAR | With USTAR | USTAR's gain |
|---|---|---|---|
| Files | 18 | | - |
| Size uncompressed | 29.810 GB | 23.198 GB | - 22.18% |
| Time | - | 7h 46m 36s | - |

*Table 8. H.Gut Reads 1*

| | Without USTAR | With USTAR | USTAR's gain |
|---|---|---|---|
| Assemblies | 3396 Genomes | | - |
| Size uncompressed | 126.79 GB | 117.12 GB | - 7.63% |
| Time | 0s (No elaboration needed) | 1D 22h 47m 21s | - |

*Table 9. H.Guts reads 2*

| | Without USTAR | With USTAR | USTAR's gain |
|---|---|---|---|
| Files | 70 | | - |
| Size uncompressed | 251.261 GB | 106.173 | - 57.74% |
| Time | - | 1g 12h 2m 22s | - |

*Table 10. H.Genome Reads*

As can be observed, the compression rates are significantly higher than those achieved on assembly-based datasets. Although the gains are not uniform across all datasets, the results clearly show that USTAR is able to effectively exploit redundancy within read data, producing substantially smaller outputs. Moreover, the downstream tools applied to the compressed datasets yield results of comparable quality to those obtained from assembly-based datasets, indicating that compression does not negatively affect correctness.
Despite these encouraging results, this approach remains unsatisfactory: the computational cost of compressing read-based datasets is too high, in some cases exceeding by orders of magnitude the time required by the considered tools to operate directly on the original data. While applying USTAR to reads may be advantageous in scenarios where storage space is the primary constraint, it does not seem to be a particularly effective use case for the tool. A more suitable and efficient application of USTAR is presented in Section 6, where the final results are reported.

## 5.3. Compression performances considerations

An analysis of the reported tables shows that USTAR's execution time does not depend solely on the total size of the dataset, as might initially be expected. Instead, performance appears to degrade as the number of files in the dataset increases. This behavior is illustrated in the graph below:

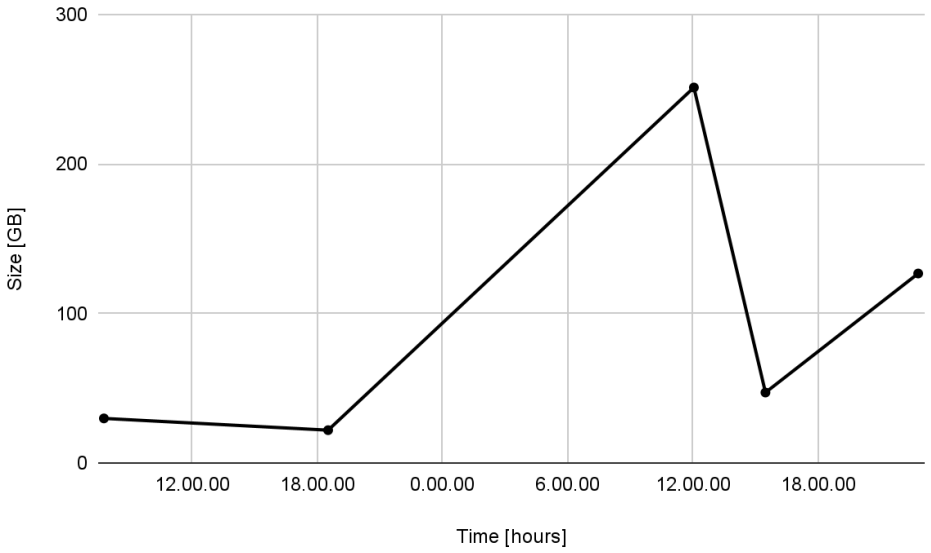


*Figure 8. USTAR's execution time is compared against the size of the dataset employed. The datasets are reads-based and assembly-based*

From the graph, it is evident that the computational time does not scale linearly with the dataset size, as hypothesized before. Although no strict quantitative correlation could be established, this behaviour could be largely attributed to the workflow of BCALM2. When USTAR is applied to a dataset composed of multiple files, the tool is executed independently on each file, requiring BCALM2 to be invoked from scratch each time. As a result, the memory allocation and graph construction steps are repeated multiple times. Even when two datasets have comparable total sizes, the repeated execution of these preprocessing steps introduces significant overhead, leading to reduced overall performance.
As shown in the following sections, operating on unitig-based datasets eliminates the need to rely on BCALM2, thereby removing this source of inefficiency and significantly improving performance consistency.

# 6. Final results

As seen in Section 5, applying USTAR-based compression to read-based or assembly-based datasets did not yield satisfactory results. After extensive experimentation, it was concluded that the most effective and advantageous way to employ USTAR is to apply it directly on unitig-based datasets, thereby bypassing BCALM2 entirely. This approach enables a substantial reduction in storage requirements while introducing only a moderate amount of computational overhead to the pipeline. Moreover, for some tools, the use of USTAR also results in faster execution times and smaller output sizes.
The results obtained from each considered tool are presented in this chapter; each tool has been applied to all datasets introduced in Section 2.

## 6.1. Analysis complications

USTAR's usefulness is maximized when applied on unitig-based datasets; however, a comparative analysis is required to properly quantify this impact. For this purpose, two versions of each dataset were initially downloaded: one based on the reads, from the NCBI SRA [16] and one based on the unitigs, from the Logan repository [4]; USTAR was then applied on the unitigs dataset to create its compressed version. Each considered tool was subsequently executed, for every dataset, on both the original read-based version and the compressed unitig-based version, with the goal of comparing performances and outputs to assess the effects of USTAR-based compression. For some tools, this two-way comparison proved insufficient to clearly interpret the results. Consequently, those tools were also applied to the uncompressed unitig-based dataset, resulting in a three-way comparison between read-based, unitig-based, and compressed unitig-based representations.

## 6.2. Analysis clarifications

The purpose of these experiments is to evaluate the performance differences introduced by applying USTAR-based compression within the analysis pipeline. As a consequence, the absolute correctness that the tools obtain starting from the uncompressed dataset is not the focus of this study. Instead, the analysis is centered on assessing how the outputs of the considered tools change when compression is introduced, in order to characterize USTAR's impact on downstream processing. Consequently, for each analysis performed in this chapter, the results produced by each tool from the original, uncompressed datasets are always considered the reference baseline, referred to as "Ground truth". As an example, in the context of a querying task, a false negative is defined as a sequence that is reported by the tool when operating on the uncompressed dataset but not reported while doing so on the compressed version. This does not imply that that sequence is necessarily present in the true biological dataset; rather, it indicates a discrepancy introduced by compression relative to the reference execution.

## 6.3. REINDEER2

REINDEER2 [6] builds an index starting from the input dataset, which can be then queried using sequences provided in fasta format. In order to compare REINDEER2's behaviour on compressed and uncompressed datasets, it is necessary to evaluate not only index construction metrics, such as size or building time, but also the responses of the indexes to queries. For this reason, a procedure for adequate query selection and comparisons was developed and is described below.

For each dataset two distinct query sets were generated, referred to A and B. Set A is derived from the read-based version of the dataset, while set B was derived from the uncompressed, unitig-based version. Each query set consists of multiple fasta files, one per query, each one containing a single randomly selected sequence extracted from a randomly chosen file of the considered dataset, preceded by its original header. In practice it is not advisable to take random sequences from a read-based dataset and use them as queries; it has been done in this thesis for the sake of consistent and fast query building but it is not advisable in a real world workflow.
The three dataset representations used in this work were then assigned numerical identifiers: the read-based version obtained from NCBI SRA is labelled as number 1, the uncompressed unitig-based version obtained from Logan is labelled as number 2 and the compressed version generated by USTAR is labelled as number 3. Comparisons were performed pairwise, with each comparison involving two indexes on a specific query set. The tested configurations were A13, B13 and B23, according to the notation introduced above. In each configuration name, the first of the two numbers corresponds to the uncompressed version of the dataset and, as discussed before, it is considered the ground truth.

### 6.3.1. Reading the results tables

The “Gain” column in the following tables follows the same notation adopted by the tables in the “Datasets” section (Section 4), where it is explained in detail. False positives and false negatives are not reported as absolute counts but as percentages. Specifically, the percentage of false negatives is computed with respect to the total number of sequences identified in the uncompressed dataset, while the percentage of false positives is computed with respect to the total number of sequences identified in the compressed dataset. What is considered to be a false positive and a false negative throughout this thesis has been described in detail in Section 6.2.

### 6.3.2. Human Genome REINDEER2 results

Index building:

| | READS | Unitigs | Compressed unitigs | Gain |
|---|---|---|---|---|
| Time | 1h 25m 9s | 22m 42s | 21m 57s | - 74.22% / - 3.30% |
| Index size | 2.405 GB | 1.997 GB | 1.997 GB | - 19.96% / - 0% |

*Table 11. Human Genome REINDEER2’s index building results*

Index querying:

| | FN | | FP | | Precision | | Recall | | F1-score | |
|---|---|---|---|---|---|---|---|---|---|---|
| | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. |
| A13 | 20% | 44.7% | 0% | 0% | 80% | 44.7% | 80% | 44.7% | 80% | 44.7% |
| B13 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |
| B23 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |

*Table 12. Human Genome REINDEER2’s query results*

### 6.3.3. Human Gut Bacteria REINDEER2 results

Index building:

| | READS | Unitigs | Compressed unitigs | Gain |
|---|---|---|---|---|
| Time | 6h 6m 31s | 50m 02s | 49m 22s | - 86.53% / - 1.33% |
| Index size | 8.018GB | 4.868 GB | 4.868 GB | - 39.28% / - 0% |

*Table 13. Human Gut Bacteria REINDEER2's index building results*

Index querying:

| | FN | | FP | | Precision | | Recall | | F1-score | |
|---|---|---|---|---|---|---|---|---|---|---|
| | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. |
| A13 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |
| B13 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |
| B23 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |

*Table 14. Human Gut Bacteria REINDEER2's query results*

### 6.3.4. Salmonella Enterica REINDEER2 results

Index building:

| | READS | Unitigs | Compressed unitigs | Gain |
|---|---|---|---|---|
| Time | 1h 47m 37s | 38m 55s | 34m 12s | - 63.84% / - 12.12% |
| Index size | 2.247 GB | 1.655 GB | 1.655 GB | - 26.35% / - 0% |

*Table 15. Salmonella REINDEER2's index building results*

Index querying:

| | FN | | FP | | Precision | | Recall | | F1-score | |
|---|---|---|---|---|---|---|---|---|---|---|
| | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. |
| A13 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |
| B13 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |
| B23 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |

*Table 16. Salmonella REINDEER2's query results*

### 6.3.5. Escherichia Coli REINDEER2 results

Index building:

| | READS | Unitigs | Compressed unitigs | Gain |
|---|---|---|---|---|
| Time | 2h 14m 49s | 59m 9s | 56m 44s | - 59.91% / - 4.09% |
| Index size | 7.778 GB | 4.801 GB | 4.801 GB | - 38.27% / - 0% |

*Table 17. E. Coli REINDEER2's index building results*

Index querying:

| | FN | | FP | | Precision | | Recall | | F1-score | |
|---|---|---|---|---|---|---|---|---|---|---|
| | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. |
| A13 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |
| B13 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |
| B23 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |

*Table 18. E. Coli REINDEER2's query results*

### 6.3.6. Drosophila Melanogaster REINDEER2 results

Index building:

| | READS | Unitigs | Compressed unitigs | Gain |
|---|---|---|---|---|
| Time | 34m 11s | 14m 55s | 14m 54s | - 56.41% / - 0.11% |
| Index size | 1.991 GB | 1.260 GB | 1.260 GB | - 36.72% / - 0% |

*Table 19. Drosophila REINDEER2's index building results*

Index querying:

| | FN | | FP | | Precision | | Recall | | F1-score | |
|---|---|---|---|---|---|---|---|---|---|---|
| | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. |
| A13 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |
| B13 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |
| B23 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |

*Table 20. Drosophila REINDEER2's query results*

### 6.3.7. REINDEER2 results analysis

The uncompressed unitig-based datasets are already smaller than the read-based ones and they become even smaller after compression. Intuitively, one would expect smaller datasets to result in faster execution times. This behaviour is indeed observed with REINDEER2; however, the gain in computational time obtained by using compressed unitigs instead of uncompressed ones is relatively modest and does not scale linearly with dataset size. By examining the resulting index sizes, it can be observed that indexes built from reads datasets are considerably larger than those

built from unitigs datasets. In contrast, indexes built from uncompressed unitigs and from USTAR-compressed unitigs are always exactly the same size. This suggests that the computational time is related to the size of the produced index rather than the size of the input alone. Moreover, the identical size of those two indexes indicates that they are always very similar to each other, a conclusion that is further supported by the query results. Consequently, compressing unitig-based datasets with USTAR does not lead to a loss of information relevant to REINDEER2 indexing.

The query comparisons further reinforce this observation. For all datasets, the B13 and B23 combinations yield perfect results, meaning that when queries are derived from unitigs, no information is lost during compression and all sequences are consistently found across all indexes.

The situation differs for the A13 combination. In this case, queries are derived from read-based datasets and thus may contain single-abundance k-mers, which are filtered in the unitig-based datasets construction. As a result, depending on the prevalence of such low-abundance k-mers in the original reads, some queries may fail entirely on both unitig-based indexes. This behaviour is particularly evident in the “Human Genome” dataset, where the statistics are significantly worse than for the other datasets, which exhibit no errors.

Reinforcing this belief, notably, in the Human Genome dataset only false negatives are observed, while the number of false positives is zero. A false negative occurs when a k-mer present in the read-based dataset is absent from the unitig-based datasets due to abundance filtering, causing the corresponding index to miss the query. Since these filtered k-mers are likely sequencing errors, this behaviour should not be interpreted as a limitation of USTAR, but rather as a natural consequence of the filtering process commonly applied during graph construction.

When queries are carefully designed to target meaningful and functional genomic regions, the k-mers they contain are expected to occur multiple times among the reads and thus will not be filtered out. In such scenarios USTAR’s compression is ideal since it doesn’t lose any information, it slightly improves performance and, most importantly, significantly reduces input size from both read-based and unitig-based datasets.

#### 6.3.7.1. REINDEER2’s statistics

Below, three graphs summarize the overall time and space gains obtained by using USTAR-compressed datasets instead of uncompressed ones. Figure 9 and Figure 10 report the speed up rates achieved when employing USTAR with respect to the reads dataset (blue) and unitigs dataset (red). Figure 9 one also accounts for the time needed to compress the unitigs dataset while the second one only considers computational times involving REINDEER2. The speed up rate indicates how many times faster using USTAR is compared to the corresponding baseline; values below one therefore indicate a slowdown when USTAR is used.

Figure 11 of the three reports the space requirements of the USTAR-based pipeline relative to both the read based (blue) and the unitig based (red) pipelines. The reported space corresponds to the sum of the input dataset size and the resulting index size. Lower values indicate greater space savings obtained through compression.

Speed up rates with respect to reads datasets are consistently high, whereas this is not the case when comparing against unitigs datasets. When the time required for compression is included, execution times are moderately worse than the unitig-based baseline; when this cost is excluded, performance improvements are just marginal. However, it is important to note that compression is a one-time operation and consequently, its computational cost can be amortized across multiple downstream analyses, significantly reducing its impact on the overall pipeline execution time.

In contrast, storage savings are consistently substantial with respect to both dataset types. As expected, storage reductions are more pronounced when starting from read-based datasets than from unitig-based ones; nevertheless, the savings remain significant.

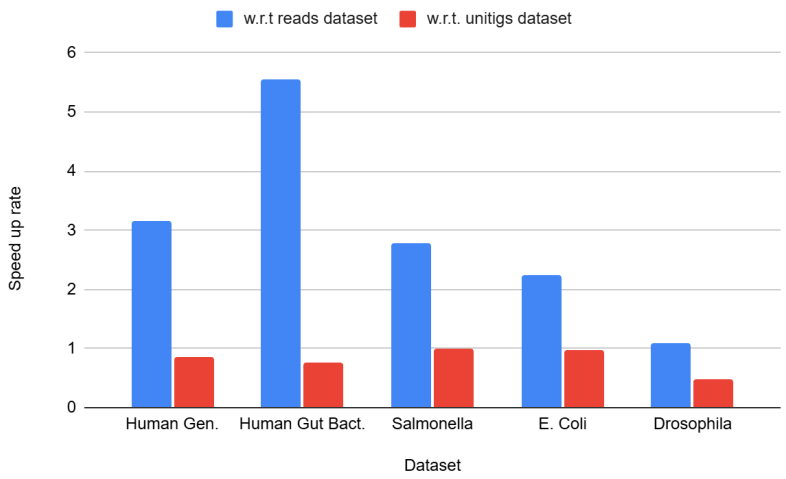


*Figure 9. USTAR's speed up rate using REINDEER2, compression time is included*

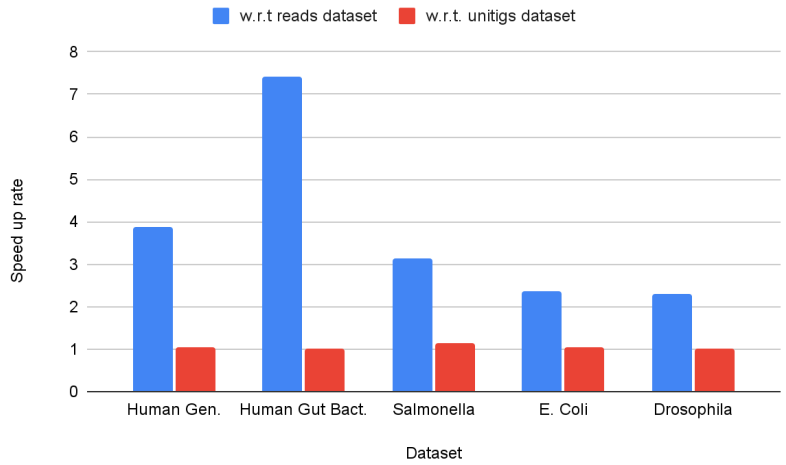


*Figure 10. USTAR's speed up rate using REINDEER2, compression time is not included*

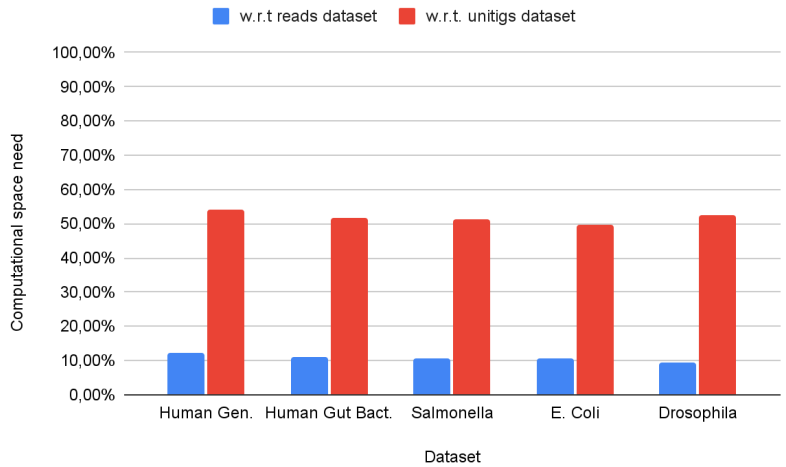


*Figure 11. Total space needed for running REINDEER2 on compressed dataset, lower is better*

## 6.4. Fulgor

Fulgor [7] is a k-mer indexing and querying tool similar to the previously described REINDEER2 [6]; however, unlike that tool, Fulgor disregards all k-mer abundance information considering instead only their binary presence/absence information. This design choice allows Fulgor to be significantly faster than other k-mer based indexing tools, including REINDEER2. The main drawback is that, by ignoring k-mer counts, Fulgor cannot filter out single-abundance k-mers and therefore relies entirely on the quality and preprocessing of the input dataset. As a consequence, comparing Fulgor's execution between uncompressed read-based dataset and compressed unitig-based datasets may reveal substantial discrepancies, since the latter are already filtered when downloading them from Logan [4], while the former are not. The query construction methodology, comparison pipeline and naming conventions adopted in this section are identical to the ones adopted in the section dedicated to REINDEER2's results (Section 6.3) and therefore are not repeated here. The same conventions apply to the labeling and interpretation of the reported tables. Refer to the REINDEERS2's results section for more information.

### 6.4.1. Human Genome Fulgor results

Index building:

| | READS | Unitigs | Compressed unitigs | Gain |
|---|---|---|---|---|
| Time | 19m 10s | 3m 31s | 3m 25s | - 82.17% / - 2.84% |
| Index size | 7.260 GB | 1.671 GB | 1.671 GB | - 76.96% / - 0% |

*Table 21. Human Genome Fulgor's index building results*

Index querying:

| | FN | | FP | | Precision | | Recall | | F1-score | |
|---|---|---|---|---|---|---|---|---|---|---|
| | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. |
| A13 | 20% | 27.4% | 24.1% | 38.2% | 75.8% | 38.2% | 80% | 27.4% | 75.4% | 36.1% |
| B13 | 11.1% | 15.7% | 0% | 0% | 100% | 0% | 88.9% | 15.7% | 93.5% | 9.3% |
| B23 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |

*Table 22. Human Genome Fulgor's query results*

### 6.4.2. Human Gut Bacteria Fulgor results

Index building:

| | READS | Unitigs | Compressed unitigs | Gain |
|---|---|---|---|---|
| Time | 1h 19m 38s | 12m 16s | 12m 16s | - 75.16% / - 0% |
| Index size | 26.423 GB | 4.453 GB | 4.453 GB | - 83.15% / - 0% |

*Table 23. Human Gut Bacteria Fulgor's index building results*

Index querying:

| | FN | | FP | | Precision | | Recall | | F1-score | |
|---|---|---|---|---|---|---|---|---|---|---|
| | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. |
| A13 | 80% | 27.4% | 92.7% | 10.1% | 7.3% | 10.1% | 20% | 27.4% | 10.7% | 14.7% |
| B13 | 14.4% | 22.1% | 6.5% | 9.3% | 93.5% | 9.3% | 85.6% | 22.1% | 89.4% | 15.9% |
| B23 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |

*Table 24. Human Gut Bacteria Fulgor's query results*

### 6.4.3. Salmonella Enterica Fulgor results

Index building:

| | READS | Unitigs | Compressed unitigs | Gain |
|---|---|---|---|---|
| Time | 25m 42s | 3m 40s | 3m 21s | - 86.96% / - 8.64% |
| Index size | 10.722 GB | 1.606 GB | 1.606 GB | - 85.02% / - 0% |

*Table 25. Salmonella Fulgor's index building results*

Index querying:

| | FN | | FP | | Precision | | Recall | | F1-score | |
|---|---|---|---|---|---|---|---|---|---|---|
| | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. |
| A13 | 80% | 27.4% | 92.7% | 10.1% | 7.3% | 10.1% | 20% | 27.4% | 10.7% | 14.7% |
| B13 | 14.4% | 22.1% | 6.5% | 9.3% | 93.5% | 9.3% | 85.6% | 22.1% | 89.4% | 15.9% |
| B23 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |

*Table 26. Salmonella Fulgor's query results*

### 6.4.4. Escherichia Coli Fulgor results

Index building:

| | READS | Unitigs | Compressed unitigs | Gain |
|---|---|---|---|---|
| Time | 1h 24m 35s | 14m 3s | 13m 10s | - 84.43% / - 6.29% |
| Index size | 29.709 GB | 5.464 GB | 5.464 GB | - 81.61% / - 0% |

*Table 27. E. Coli Fulgor's index building results*

Index querying:

| | FN | | FP | | Precision | | Recall | | F1-score | |
|---|---|---|---|---|---|---|---|---|---|---|
| | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. |
| A13 | 10% | 22.4% | 10% | 22.4% | 90% | 22.4% | 90% | 22.4% | 90% | 22.4% |
| B13 | 21.7% | 33.6% | 11.7% | 21.7 | 88.3% | 21.7% | 78.3% | 33.6% | 81.8% | 30.1% |
| B23 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |

*Table 28. E. Coli Fulgor's query results*

### 6.4.5. Drosophila Melanogaster Fulgor results

Index building:

| | READS | Unitigs | Compressed unitigs | Gain |
|---|---|---|---|---|
| Time | 27m 51s | 3m 12s | 3m 08s | - 88.75% / - 2.08% |
| Index size | 8.281 GB | 1.211 GB | 1.211 GB | - 85.38% / - 0% |

*Table 29. Drosophila Fulgor's index building results*

Index querying:

| | FN | | FP | | Precision | | Recall | | F1-score | |
|---|---|---|---|---|---|---|---|---|---|---|
| | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. | avg. | st.dev. |
| A13 | 50% | 50% | 50% | 50% | 50% | 50% | 50% | 50% | 50% | 50% |
| B13 | 26.7% | 36.5% | 10% | 22.4% | 90% | 22.4% | 73.3% | 36.5% | 78% | 30.3% |
| B23 | 0% | 0% | 0% | 0% | 100% | 0% | 100% | 0% | 100% | 0% |

*Table 30. Drosophila Fulgor's query results*

### 6.4.6. Fulgor results analysis

Similarly to REINDEER2, employing Fulgor on compressed datasets slightly improves computation times with respect to uncompressed unitg-based datasets. As observed for REINDEER2, these time gains likely occur due to the smaller storage footprint of the compressed input, even if they don't seem to scale linearly with its size.

The indexes produced by Fulgor from read-based datasets are significantly larger than those built from unitig-based datasets. In contrast, indexes built from uncompressed and compressed unitigs always have the same size. This suggests that the two unitig-based indexes are nearly identical and carry essentially the same information. This observation is further supported by the query results, which are always perfect in the B23 configuration, indicating no meaningful differences between indexes 2 and 3.

Configuration A12 and B13 instead yield suboptimal results: fulgor's inability to filter out single occurrence k-mers introduces noticeable differences in k-mer presence across datasets, which negatively affects query performance. False negatives are straightforward to explain: read-based datasets contain many single-occurrence k-mers, which are filtered from the unitig-based ones.

This means that, in the read-based index, a match for a query may be found on a sequence containing one of these k-mers, while it can not be found in the unitig-based version of that dataset, since those k-mers have been filtered out. False positives are more difficult to interpret; one possible explanation is related to the removal of k-mers during filtering: when a k-mer is removed, the two remaining sequence's ends are effectively "stitched together", forming a new sequence that was not present in the original dataset, that could spuriously match a query. In this case, using longer queries would likely mitigate the issue. Alternatively, false positives may be related to the inner workings of Fulgor's indexing strategy. In any case, investigating the exact cause of these false positives is outside the scope of this thesis and can be considered as future work. In practice, input datasets used in biological experiments are typically filtered and therefore, these issues should not arise; the observed discrepancies are mainly a consequence of the artificial comparison between filtered and unfiltered datasets performed in this analysis and should not appear in real-word scenarios.

#### 6.4.6.1. Fulgor statistics

Below, three graphs summarize the overall time and space gains obtained by using USTAR-compressed datasets instead of uncompressed ones. The reported figures follow the same structure as the ones reported in the "REINDEER2's statistics" section (Section 6.3.7.1) where they are thoroughly explained.

Speed up rates with respect to reads are consistently high, reaching upwards of being almost 9 times faster while the compression time is not taken into account. In contrast, speed up rates with respect to unitigs datasets are small or less than one. Without accounting for the compression time, Fulgor is only marginally faster when operating on the compressed version of the dataset; however, once compression time is included, execution times favor the uncompressed unitig-based pipeline. As mentioned for REINDEER2, compression is a one-time operation whose cost can be amortized across multiple downstream analyses, substantially reducing its impact on the overall execution time. It can also be observed that, when accounting for the compression time, speed up rates with respect to the unitigs dataset are lower than the analogues in REINDEER2. This behavior is expected given Fulgor's faster execution times: since the compression cost is independent of the downstream tool, its relative weight increases as the execution time of the indexing tool decreases, thereby reducing the overall speedup.

Regarding storage requirements, the compressed datasets require consistently little time compared to both reads and unitigs datasets. In particular, the total storage needed when using compressed unitigs is always below 10% of that required by read-based datasets and around 50% of that required by unitig-based datasets.

Overall, these results empathize how much storage can be saved by choosing to adopt USTAR into the pipeline for a, depending on the perspective, almost non-existent performance penalty.

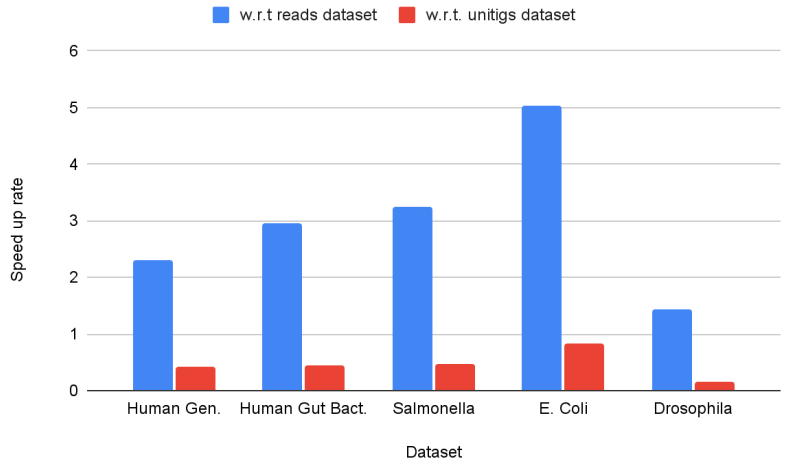


*Figure 12. USTAR's speed up rate using Fulgor, compression time is included*

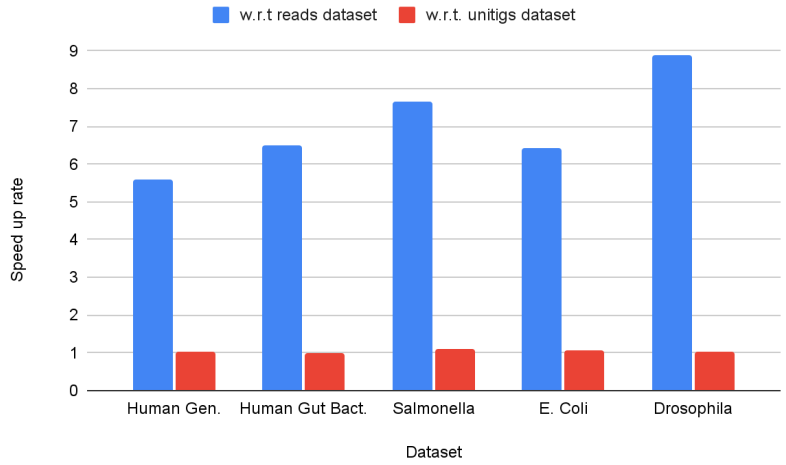


*Figure 13. USTAR's speed up rate using Fulgor, compression time is not included*

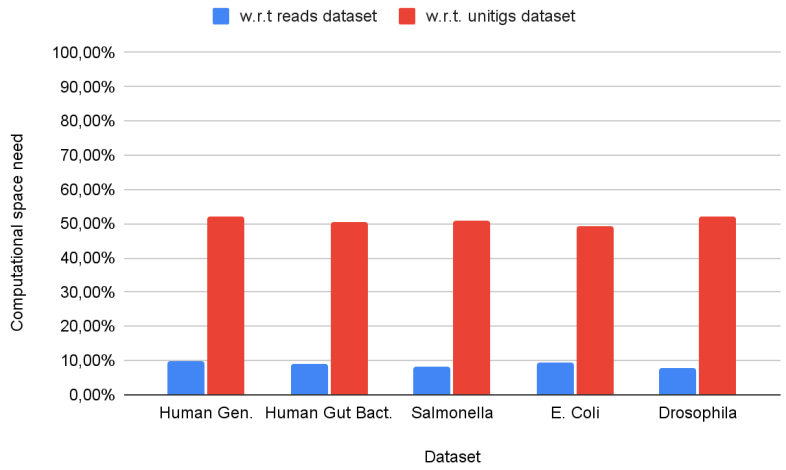


*Figure 14. Total space needed for running Fulgor on compressed dataset, lower is better*

## 6.5. GGCAT

GGCAT [9] is a tool for building compacted, and optionally colored, De Bruijn Graphs; it also supports the merging of multiple DBGs and enables querying. Although GGCAT is not strictly an indexing tool, it can be used as such by querying the DBGs created from the input sequences. This versatility allows GGCAT to be employed as part of the REINDEER2's [6] pipeline, where GGCAT is responsible for constructing the graphs while REINDEER2 performs efficient k-mer indexing on top of them. Consequently, in the REINDEER2 performance analysis reported in Section 6.3, GGCAT's execution time is included as part of the REINDEER2's overall computational cost.
The evaluation of GGCAT's performances in this thesis considers aspects analogous to those considered for the previously described tools, namely execution time and storage requirements.

### 6.5.1. GGCAT's query performances and analysis clarifications

Queries used with GGCAT were prepared and employed exactly as for the previously analyzed tools; for an in-depth description of the query construction and comparison methodology, refer to the REINDEER2's section Section 6.3.
Unlike the tools discussed earlier, the initial experiments with GGCAT immediately showed very strong query performances. In fact, all the queries for each of the previously defined comparison configurations (Section 6.3) yielded perfect results, with F1-score always equal to one. For this reason, tables reporting query accuracy metrics are omitted in this section, as they would only contain perfect scores for all statistics. Given the consistently high quality of the results, the three-way comparison strategy adopted in earlier analyses, and described in Section 6.1 was deemed unnecessary for GGCAT. Consequently, only the reads datasets and their USTAR-compressed unitig-based counterparts are compared in this section.
The following tables are formatted identically to those presented for the previous two tools, with the only difference being the absence of the "Unitigs" column due to the exclusion of the three-way comparison. For a detailed explanation of the table structure and reported metrics, refer to the dedicated subsection in the REINDEER2 section (Section 6.3.1).

### 6.5.2. Human Genome GGCAT results

Graph building:

| | READS | Compressed unitigs | Gain |
|---|---|---|---|
| Time | 59m 03s | 14m 39s | - 75.19% |
| Graph size | 8.085 GB | 4.735 GB | - 41.43% |

*Table 31. Human Genome GGCAT's graph building results*

### 6.5.3. Human Gut Bacteria GGCAT results

Graph building:

| | READS | Compressed unitigs | Gain |
|---|---|---|---|
| Time | 4h 52m 6s | 30m 47s | - 89.46% |
| Graph size | 34.775 GB | 11.433 GB | - 67.12% |

*Table 32. Human Gut Bacteria GGCAT's graph building results*

### 6.5.4. Salmonella Enterica GGCAT results

Graph building:

| | READS | Compressed unitigs | Gain |
|---|---|---|---|
| Time | 1h 12m 9s | 34m 24s | - 52.32% |
| Graph size | 8.888 GB | 4.983 GB | - 43.94% |

*Table 33. Salmonella GGCAT's graph building results*

### 6.5.5. Escherichia Coli GGCAT results

Graph building:

| | READS | Compressed unitigs | Gain |
|---|---|---|---|
| Time | 49m 41s | 8m 01s | - 83.87% |
| Graph size | 35.683 GB | 13.729 GB | - 61.53% |

*Table 34. E. Coli GGCAT's graph building results*

### 6.5.6. Drosophila Melanogaster GGCAT results

Graph building:

| | READS | Compressed unitigs | Gain |
|---|---|---|---|
| Time | 12m 24s | 2m 03s | - 83.47% |
| Graph size | 8.617 GB | 3.730 GB | - 64.16% |

*Table 35. Drosophila GGCAT's graph building results*

### 6.5.7. GGCAT results analysis

GGCAT is a component of the REINDEER2 pipeline and its execution times are generally lower (within a margin of error) than the corresponding execution times observed for REINDEER2 on the same datasets. This behavior is expected, as REINDEER2 first relies on GGCAT to process the input dataset and then performs additional computations on the resulting graphs, introducing computational overhead. This extra processing is required to build an efficient k-mer index, which is not the primary objective of GGCAT. Consistently with this design, REINDEER2 produces output files that are smaller than those generated by GGCAT, as they are specifically optimized for compact index size and fast query performance.

Overall computational time gains are comparable to the ones observed for the previously analyzed tools: they are substantial and represent a significant advantage of employing USTAR-based compression. Storage space savings are even more pronounced than they were in REINDEER2, although they are less aggressive than they were on Fulgor. It can be seen that GGCAT's output size drops faster with the reduction of input size when compared to REINDEER2, but slower when compared to Fulgor. In all cases, the reduction in output size achieved by applying USTAR-based compression remains consistently significant.

As reported in Section 6.5.1, query results are always perfect when filtering the read-based dataset for a minimum k-mer abundance of two. Consequently, for all considered datasets, no information

loss was observed when comparing uncompressed read-based datasets with their USTAR-compressed unitig-based counterparts.

#### 6.5.7.1. GGCAT's statistics

Below, three graphs summarize the overall time and space gains obtained by using USTAR-compressed datasets instead of uncompressed ones. The reported figures follow the same structure as the ones reported in Section 6.3.7.1, where an in depth explanation is present.
Comparing the figures reported below with those obtained from REINDEER2 (of which GGCAT is a part of), it can be immediately observed that computational time gains are significantly higher for GGCAT, both when accounting for compression time (Figure 15) and when excluding it (Figure 16). In particular, as seen in the first figure below, and differently from REINDEER2, the speed up rates are greater than one for almost all datasets. This behavior is a direct consequence of the additional computational overhead introduced by REINDEER2 on top of GGCAT, which slows down execution times. As previously discussed, this overhead allows REINDEER2 to produce more compact outputs, specifically optimized for k-mer indexing and querying. This trade-off is evident when comparing GGCAT's storage reduction graph (Figure 17) with the corresponding one for REINDEER2: the latter shows how REINDEER2 requires considerably less space to accommodate the entire pipeline.
Independently of comparisons with REINDEER2, both execution time and storage requirements for GGCAT benefit substantially from the USTAR-based compression of the original read-based datasets. The consistent improvements observed across all datasets indicate that USTAR effectively reduces input redundancy while preserving the structural information required by GGCAT to construct accurate compacted De Bruijn Graphs. These results further confirm that USTAR performs well in practice and can be successfully integrated upstream of graph-based tools without negatively impacting their performance or correctness.

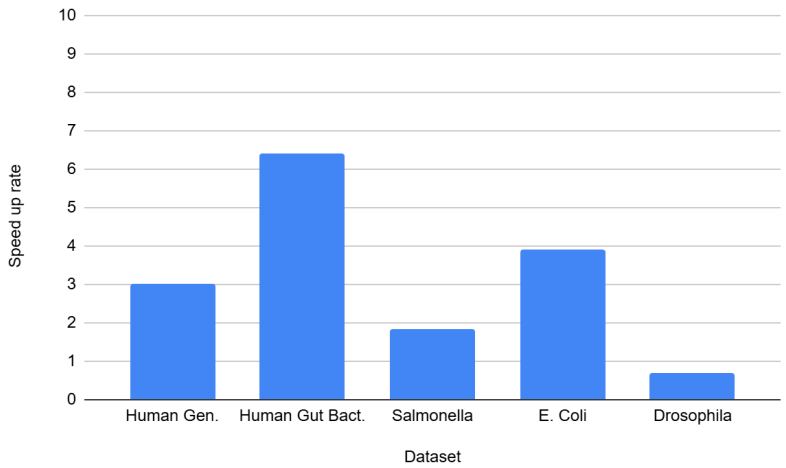


*Figure 15. USTAR's speed up rate using GGCAT, compression time is included*

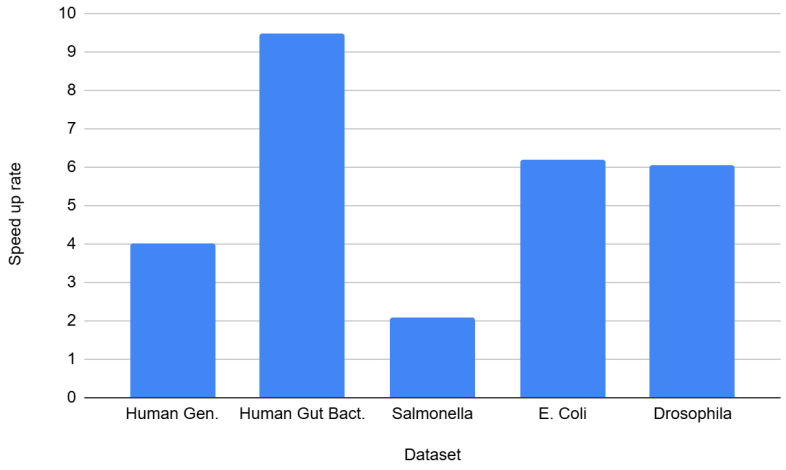


*Figure 16. USTAR's speed up rate using GGCAT, compression time is not included*

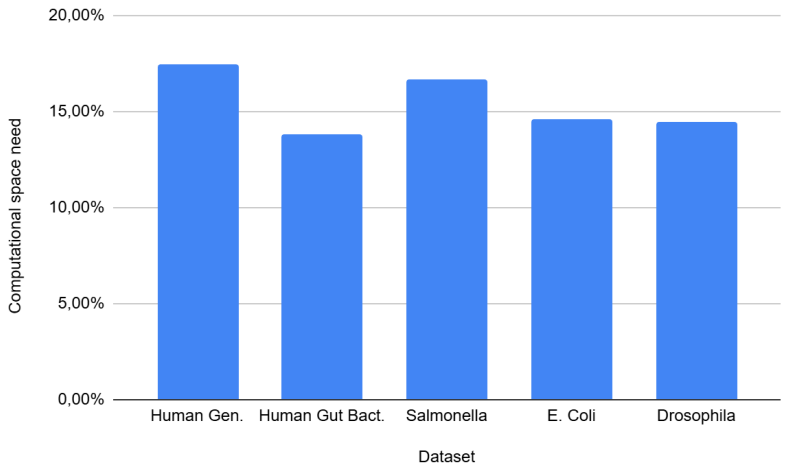


*Figure 17. Total space needed for running GGCAT on compressed dataset, lower is better*

## 6.6. Mash

In the context of this thesis, as mentioned in Section 2.3, Mash [8] was used to estimate distances between genomes by first reducing them into compact sketch representations. As previously discussed for GGCAT Section 6.5.1, Mash exhibited strong performance when applied directly to read-based datasets. For this reason, only a two-way comparison between uncompressed read-based datasets and USTAR-compressed unitig-based datasets was performed. To conduct the analysis, for both the compressed and uncompressed versions of each dataset, every file was first reduced to its sketch representation. Subsequently, for each dataset version, pairwise distances between all files were computed using Mash. This procedure produces a N × N matrix of distance values ranging between 0 and 1, where values close to 0 indicate highly dissimilar sequences and values close to 1 indicate identical sequences. If USTAR-based compression preserved all relevant sequence information, the resulting distance matrices for the compressed and uncompressed datasets would have identical cells. However, since it is not possible to compress genomic sequences without incurring some degree of information loss, a certain amount of discrepancy between the two matrices is expected. To ensure that the observed differences are primarily attributable to USTAR's compression rather than to sequencing noise, single-abundance k-mers were filtered from the read-based datasets, minimizing the impact of sequencing errors.

### 6.6.1. Mash analysis results explanation

The results of the Mash analysis are reported, for each dataset, in two tables. The first table describes the computational time required to execute the tool, while the second summarizes the differences between the distance matrices obtained from the read-based datasets and their USTAR-compressed unitig-based counterparts. To quantify the differences between these distance matrices, and consequently to assess the impact introduced by USTAR, two statistical plots were generated for each dataset: a *Bland-Altman plot* and a *correlation plot*. The corresponding summary statistics derived from these plots are reported in the "Plots" tables.
The Bland-Altman plot is used to assess the agreement between two measurement methods. In this context, it highlights systematic biases and the dispersion of differences between distances computed on compressed and uncompressed datasets. The correlation plot, on the other hand, is used to evaluate the strength of the association between the two sets of distance values. Both the Pearson correlation coefficient ($r$), which measures linear correlation, and the Spearman rank correlation coefficient ($\rho$), which assesses monotonic relationships, are reported for each dataset.

### 6.6.2. Human Genome Mash results

Execution:

| | READS | Compressed unitigs | Gain |
|---|---|---|---|
| Time | 9m 17s | 26s | - 95.33% |

*Table 36. Human Genome Mash's execution results*

Plots:

| Bland-Altman plot | | | Correlation plot | |
|---|---|---|---|---|
| Mean difference | Upper LoA | Lower LoA | Pearson *r* | Spearman *ρ* |
| 0.0095 | 0.0419 | - 0.0229 | 0.9909 | 0.9744 |

*Table 37. Human Genome Mash plots results*

### 6.6.3. Human Gut Bacteria Mash results

Execution:

| | READS | Compressed unitigs | Gain |
|---|---|---|---|
| Time | 41m 43s | 1m 2s | - 97.52% |

*Table 38. Human Gut Bacteria Mash's execution results*

Plots:

| Bland-Altman plot | | | Correlation plot | |
|---|---|---|---|---|
| Mean difference | Upper LoA | Lower LoA | Pearson *r* | Spearman *ρ* |
| 0.0122 | 0.1210 | - 0.0966 | 0.9826 | 0.9934 |

*Table 39. Human Gut Bacteria Mash plots results*

### 6.6.4. Salmonella Enterica Mash results

Execution:

| | READS | Compressed unitigs | Gain |
|---|---|---|---|
| Time | 14m 5s | 32s | - 96.21% |

*Table 40. Salmonella Mash's execution results*

Plots:

| Bland-Altman plot | | | Correlation plot | |
|---|---|---|---|---|
| Mean difference | Upper LoA | Lower LoA | Pearson *r* | Spearman *ρ* |
| 0.0059 | 0.1270 | - 0.1153 | 0.9661 | 0.9945 |

*Table 41. Salmonella Mash plots results*

### 6.6.5. Escherichia Coli Mash results

Execution

| | READS | Compressed unitigs | Gain |
|---|---|---|---|
| Time | 38m 35s | 27s | - 98.83% |

*Table 42. E. Coli Mash's execution results*

Plots:

| Bland-Altman plot | | | Correlation plot | |
|---|---|---|---|---|
| Mean difference | Upper LoA | Lower LoA | Pearson *r* | Spearman *ρ* |
| 0.0093 | 0.1068 | - 0.0882 | 0.9590 | 0.9952 |

*Table 43. E. Coli Mash plots results*

### 6.6.6. Drosophila Melanogaster Mash results

Execution

| | READS | Compressed unitigs | Gain |
|---|---|---|---|
| Time | 9m 40s | 14s | - 97.58% |

*Table 44. Drosophila Mash's execution results*

Plots:

| Bland-Altman plot | | | Correlation plot | |
|---|---|---|---|---|
| Mean difference | Upper LoA | Lower LoA | Pearson *r* | Spearman *ρ* |
| 0.0069 | 0.0820 | - 0.0681 | 0.9875 | 0.9947 |

*Table 45. Drosophila Mash plots results*

### 6.6.7. Mash results analysis

The computational time gain is substantial: among all the tools considered in this thesis, Mash exhibits the largest reduction in execution time when operating on USTAR-compressed datasets. For every dataset, the reduction in computational time is consistently above 95%.
Considering that all values in the distance matrix range between 0 and 1, the observed mean differences are particularly small. The upper and lower *Limits of Agreements* (LoAs) are less pronounced, although still relatively small when compared to the full range of possible values. This behavior is largely attributable to the presence of a limited number of outliers in each dataset, which disproportionately affect the LoAs. These outliers may be caused by differences between Mash's internal k-mer filtering strategy and the filtering applied to the Logan unitigs prior to USTAR compression. Alternatively, they may arise from specific aspects of Mash's internal sketching mechanisms. In any case, an investigation into the precise origin of these outliers is beyond the scope of this thesis. Conclusions consistent with the first plot can be drawn from the correlation analysis. Both the Pearson correlation coefficient (*r*) and the Spearman rank correlation coefficient (*ρ*) are consistently close to one across all datasets, indicating a strong correlation between distances computed on compressed and uncompressed datasets. These results suggest that the correctness of Mash-based analyses is not significantly affected by USTAR's compression, which can therefore be safely employed in most practical scenarios, with the exception of applications in which even minimal numerical deviations are not acceptable.

### 6.6.7.1. Mash's statistics

Below, three graphs summarize the overall time and space gains obtained by using USTAR-compressed datasets instead of uncompressed ones. The reported figures follow the same structure as the ones reported in Section 6.3.7.1; refer to it for an in depth explanation.

The speed up rates are very high. When accounting for compression time (Figure 18), they are greater than one for almost every datasets, indicating that applying USTAR results in an overall faster pipeline. Moreover, the observed speedups are higher than those obtained for all the other considered tools. When compression time is not accounted for (Figure 19), the speedup rates increase dramatically, reaching values of up to 85× compared to the uncompressed pipeline.

The Mash distance output matrix is overall of a negligible size; as a consequence, in Figure 20, the storage requirements of Mash are almost entirely determined by the size of the input datasets themselves. Mash does not require the construction of large auxiliary data structures on top of the input data, which allows it to maintain proportionally lower storage requirements than the other analyzed tools. This characteristic makes Mash particularly well suited to benefit from USTAR-based compression.

Overall, the substantial gains in computational time combined with the significant storage savings achieved by using USTAR in conjunction with Mash, demonstrate that Mash is highly compatible with USTAR-compressed datasets, making it one of the most effective applications of USTAR evaluated in this work.

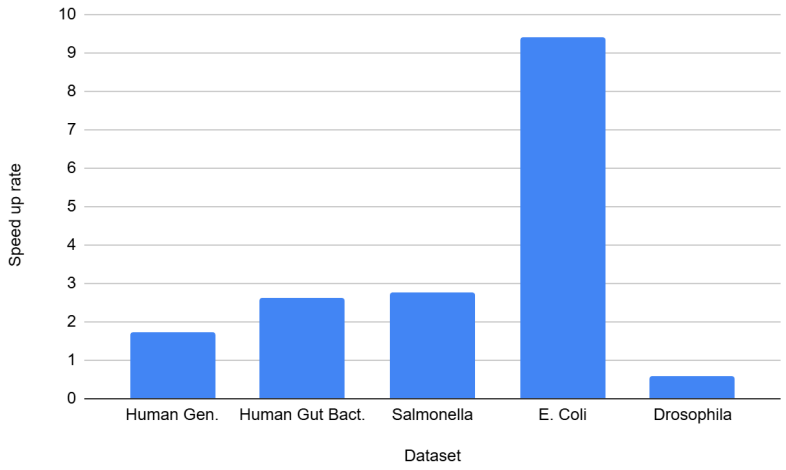


*Figure 18. USTAR's speed up rate using Mash, compression time is included*

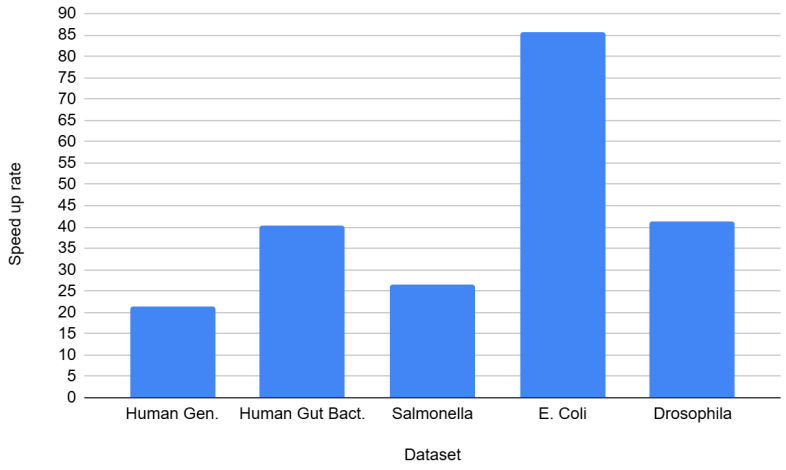


*Figure 19. USTAR's speed up rate using Mash, compression time is not included*

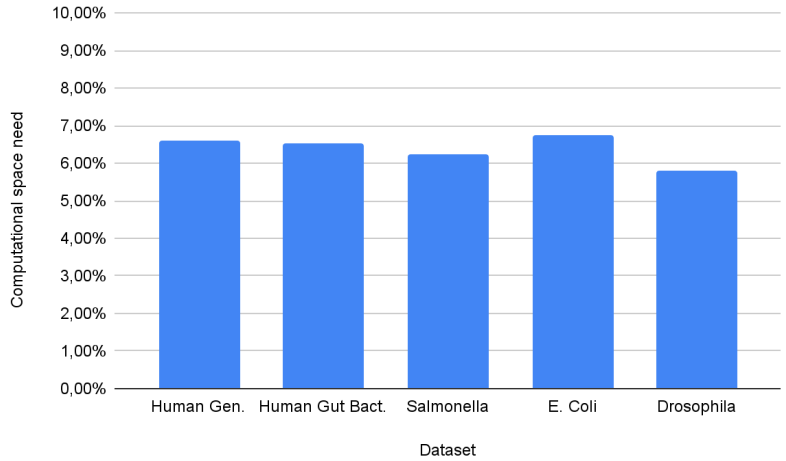


*Figure 20. Total space needed for running Mash on compressed dataset, lower is better*

## 6.7. Results considerations

Below, two tables summarizing the results obtained from each tool are reported. Table 46 shows the speedup obtained by each tool on each dataset, defined as how many times faster the tool executes when operating on compressed, unitig-based datasets compared to uncompressed, read-based ones. Since compression is performed only once, and the resulting compressed dataset can be reused by multiple tools, the more tools are applied to the same dataset, the less relevant the one time compression cost becomes to the overall pipeline. For this reason, explicitly accounting for compression time in the reported table would not be fully coherent with the intended analysis; compression times are thus not part of Table 46. Nevertheless, for completeness sake, plots accounting for compression times have been reported in each of the tool-specific results sections. As shown in the table, the observed speedup rates are significant: for every tool and every dataset, using compressed, unitig-based datasets instead of the corresponding uncompressed, read-based datasets, results in at least a two times reduction of computational time, with many cases showing significantly larger improvements.
Table 47 reports the storage requirements of each tool's pipeline when operating on compressed unitig-based datasets, expressed as a percentage of the storage required by the same tools when operating on uncompressed, read-based datasets. In all cases, the required storage is well below 20% of the baseline, and for several tools it drops below 10%.
Considering both tables, all the tested tools show strong performance improvements, fully justifying the adoption of USTAR's compression within their pipelines. Particularly, among them, Mash clearly stands out, achieving substantially better results than the other tools in terms of both computational time reduction and storage savings.
Overall, the accuracy of the results obtained when using USTAR-compressed datasets is more than sufficient for the vast majority of applications involving the analyzed tools, with the only notable exception being Fulgor. In particular, Fulgor's results are affected by the lack of a built-in mechanism to filter k-mers based on abundance; this means that Fulgor must run on pairs of datasets of which only one of the two has been filtered, causing discrepancies in the results. These discrepancies are thus not intrinsic to USTAR itself; rather, they arise from this comparison between these differently-filtered datasets, a comparison that is purely artificial and performed in this thesis for the sake of evaluating tools' performances. In principle, since Fulgor ignores abundance information entirely, the information removed by USTAR's compression should not affect its results, potentially leading to perfect agreement. Investigating this behavior in a more controlled setting is beyond the scope of this thesis but represents a natural direction for future work.

| | REINDEER2 | Fulgor | GGCAT | Mash |
|---|---|---|---|---|
| Human Genome | 3.88 | 5.61 | 4.03 | 21.42 |
| H. gut Bacteria | 7.42 | 6.49 | 9.49 | 40.37 |
| Salmonella | 3.15 | 7.67 | 2.10 | 26.41 |
| E. Coli | 2.38 | 6.42 | 6.20 | 85.74 |
| Drosophila | 2.29 | 8.89 | 6.05 | 41.43 |

*Table 46. Speed up rates of every tool for each compressed dataset w.r.t. reads, compression is not included, higher is better*

| | REINDEER2 | Fulgor | GGCAT | Mash |
|---|---|---|---|---|
| Human Genome | 12.18% | 9.76% | 17.47% | 6.61% |
| H. gut Bacteria | 10.89% | 8.84% | 13.79% | 6.51% |
| Salmonella | 10.48% | 8.37% | 16.68% | 6.25% |
| E. Coli | 10.73% | 9.25% | 14.58% | 6.76% |
| Drosophila | 9.53% | 7.77% | 14.45% | 5.80% |

*Table 47. Storage requirements of every tool for each compressed dataset w.r.t reads, initial dataset size is included, lower is better*

# 7. Conclusions and future works

As previously discussed in Section 5.7, the experimental results show that integrating USTAR into k-mer based pipelines provides multiple advantages, notably substantially reduced execution times and significantly lower storage requirements. The cost of compression itself is relatively small and, since it is a one-time operation, it can be effectively amortized by reusing the same compressed dataset across multiple downstream tools, progressively reducing its impact on the overall pipeline.
Although the results obtained in this thesis are overall satisfactory, several directions for future work could further clarify and extend the presented findings. For instance, a deeper investigation into the origin of the outliers observed in the Mash distance matrices would be valuable, as well as understanding why some datasets exhibit lower accuracy than others. In this thesis, only hypotheses were proposed for these behaviors, which should be validated through dedicated and more controlled experiments. Another relevant extension would be the adoption of a tool-independent k-mer abundance filtering strategy, to be applied uniformly across all datasets. This would enable more direct and fair comparisons between tools and would likely improve tools' results for the tools that do not include native abundance filtering mechanisms, such as Fulgor.
Due to the internal scheduling and resource management of the DEI cluster, it was not possible to reliably and precisely measure RAM usage of the tested tools when operating on compressed and uncompressed datasets. Accurately assessing memory consumption represents a potentially important future work, as it could reveal additional benefits of USTAR-based compression beyond execution time and storage savings. Further insights could also be gained by evaluating USTAR on larger and more heterogeneous datasets, as well as by employing longer and more diverse queries. Finally, extending the experimental evaluation to additional k-mer based tools would further strengthen and generalize the conclusions drawn from this work.
In conclusion, USTAR proved to be a robust and effective tool capable of significantly improving performances across a wide range of k-mer based bioinformatics applications. By enabling faster execution times and reduced storage requirements with minimal impact on result accuracy, USTAR represents a solid foundation and a valuable starting point towards more efficient and scalable bioinformatics workflows.

# 8. Acknowledgments

I would like to thank my supervisor Comin Matteo for his guidance, availability, and continuous support throughout the development of this thesis. His feedback and suggestions were essential in shaping this work. I would also like to thank the University of Padova and the Department of Information Engineering for providing the computational infrastructure and server resources necessary to carry out the experimental analyses presented in this thesis. Finally, I would like to thank my family for their constant encouragement and support throughout all my academic journey as well as during the writing of this thesis.

# 9. Bibliography

[1] Rossignolo, E., and M. Comin. "USTAR: Improved Compression of k-mer Sets with Counters Using de Bruijn Graphs." Bioinformatics Research and Applications. ISBRA 2023. A cura di X. Guo, S. Mangul, M. Patterson, and A. Zelikovsky, vol. 14248, Springer, 2023, doi:10.1007/978-981-99-7074-2_16.
[2] Orsolon, Ludovico. MasterThesis. GitHub, 2025, www.github.com/OrsolonLudovico/MasterThesis.
[3] Chikhi, Rayan, Antoine Limasset, and Paul Medvedev. "Compacting de Bruijn graphs from sequencing data quickly and in low memory." Bioinformatics 32.12 (2016): i201-i208.
[4] Chikhi, Rayan, et al. "Logan: planetary-scale genome assembly surveys life's diversity." bioRxiv (2024): 2024-07.
[5] Marchet Picy, Camille, et al. "REINDEER: efficient indexing of k-mer presence and abundance in sequencing datasets." (2020).
[6] Hernandez–Courbevoie, Yohan, et al. "REINDEER2: practical abundance index at scale." International Symposium on String Processing and Information Retrieval. Cham: Springer Nature Switzerland, 2025.
[7] Fan, Jason, et al. "Fulgor: a fast and compact k-mer index for large-scale matching and color queries." Algorithms for Molecular Biology 19.1 (2024): 3.
[8] Ondov, Brian D., et al. "Mash: fast genome and metagenome distance estimation using MinHash." Genome biology 17.1 (2016): 132.
[9] Cracco, Andrea, de Bruijn graphs with GGCAT." Genome Research 33.7 (2023): 1198-1207.
[10] Ranallo-Benavidez, T. Rhyker, Kamil S. Jaron, and Michael C. Schatz. "GenomeScope 2.0 and Smudgeplot for reference-free profiling of polyploid genomes." Nature communications 11.1 (2020): 1432.
[11] Mapleson, Daniel, et al. "KAT: a K-mer analysis toolkit to quality control NGS datasets and genome assemblies." Bioinformatics 33.4 (2017): 574-576.
[12] "Blade" computing cluster. Unipd, 2025, www.dei.unipd.it/en/node/2284
[13] Slurm Workload Manager. ShedMD, 2025 www.slurm.schedmd.com
[14] SingularityCE. Syllabus, 2025 www.docs.sylabs.io/guides/latest/user-guide
[15] Conway, Thomas C., and Andrew J. Bromage. "Succinct data structures for assembling large genomes." Bioinformatics 27.4 (2011): 479-486.
[16] National Center for Biotechnology Information. National Library of Medicine 2025 www.ncbi.nlm.nih.gov
[17] Broder, Andrei Z. "On the resemblance and containment of documents." Proceedings. Compression and Complexity of SEQUENCES 1997 (Cat. No. 97TB100171). IEEE, 1

# 10. Appendix

## 10.1. Accessions lists

For each of the datasets presented in Section 4, the list of the accessions used for that dataset is reported here.

### 10.1.1. Human Genome

ERR1481233 ERR3895557 SRR12085034 SRR12092909 SRR12421211 SRR12743666 SRR12819912 SRR14575579 SRR15630300 SRR20806281 SRR22473183 SRR3493739 SRR5825395

### 10.1.2. Human Gut bacteria

DRR152401 ERR027580 ERR1264968 ERR1332122 ERR1672989 ERR1673611 ERR2009395 ERR4079873 ERR4969496 ERR576072 SRR10036683 SRR12453064 SRR13920124 SRR13930863 SRR14763373 SRR15655001 SRR16951193 SRR17253026 SRR18274666 SRR18386946 SRR18921011 SRR19203994 SRR19251336 SRR21389197 SRR23120148 SRR23670082 SRR24986849 SRR26935188 SRR4227937 SRR490497 SRR5545435 SRR5845029 SRR6966619 SRR7486183 SRR7681786 SRR7762102 SRR8063359 SRR9660035

### 10.1.3. Salmonella enterica

DRR013735 DRR015209 DRR109429 DRR132661 DRR136121 DRR152144 DRR176354 DRR190846 DRR192979 DRR263033 DRR282342 DRR322529 DRR379098 DRR394779 ERR001002 ERR001557 ERR002168 ERR015625 ERR027713 ERR027889 ERR059428 ERR097162 ERR10019034 ERR1110399 ERR1281308 ERR1321181 ERR6053041 SRR13750715 SRR15078490 SRR17903276 SRR20784516 SRR21856049 SRR22070390 SRR23853795 SRR24229492 SRR7554384 SRR8546091 SRR9836259

### 10.1.4. Escherichia Coli

ERR1265725 ERR2267853 ERR3011891 ERR3267358 ERR4139995 ERR4984553 SRR036613 SRR12188450 SRR12224376 SRR12333695 SRR13158204 SRR15135230 SRR17466673 SRR19250702 SRR22843828 SRR23974366 SRR24072488 SRR24809659 SRR24991714 SRR26226479 SRR26454041 SRR28067249 SRR28785065 SRR30248701 SRR31510420 SRR32816012 SRR33592055 SRR34062964 SRR5203558 SRR604603 SRR6785706 SRR6849994 SRR9671851

### 10.1.5. Drosophila melanogaster

DRR013735 DRR015209 DRR109429 DRR132661 DRR136121 DRR152144 DRR176354 DRR190846 DRR192979 DRR263033 DRR282342 DRR322529 DRR379098 DRR394779 ERR001002 ERR001557 ERR002168 ERR015625 ERR027713 ERR027889 ERR059428 ERR097162 ERR10019034 ERR1110399 ERR1281308 ERR1321181 ERR6053041 SRR13750715 SRR15078490 SRR17903276 SRR20784516 SRR21856049 SRR22070390 SRR23853795 SRR24229492 SRR7554384 SRR8546091 SRR9836259

## 10.2. Parameters employed

All the scripts used in this thesis are reported on the dedicated GitHub page [2]. Inside those scripts are specified the parameters used for each tool, in each application. Refer to that page for more information about the implementation of the employed tools.

## 10.3. Data used to draw the images and graphs

In this section lies all the data used to build each graph in this thesis, divided by chapters. Each table refers to a specific figure in this thesis.

### 10.3.1. Initial results

Figure 8

| Name | Size [GB] | # files | Time |
|---|---|---|---|
| Human Gut Reads | 29,81 | 18 | 7.46.36 |
| Salm1 Ass | 21,94 | 4589 | 18.31.00 |
| Human Gen Reads | 251,261 | 70 | 36.02.22 |
| Salm2 Ass | 47,13 | 10000 | 39.28.00 |
| Human Gut Ass | 126,79 | 3396 | 46.47.00 |

### 10.3.2. Final results

#### 10.3.2.1. REINDEER2

Figure 9

| **Time** | Total time Reads | total time unitigs | total time comp | speedup rate wrt reads | speedup rate wrt unitigs |
|---|---|---|---|---|---|
| Human Gen. | 1h 25m 9s | 22m 42s | 26m 53s | 3,1674 | 0,8444 |
| Human Gut Bact. | 6h 6m 31s | 50m 02s | 66m 08s | 5,5421 | 0,7566 |
| Salmonella | 1h 47m 37s | 38m 55s | 38m 45s | 2,7772 | 1,0043 |
| E. Coli | 2h 14m 49s | 59m 9s | 60m 23s | 2,2327 | 0,9796 |
| Drosophila | 34m 11s | 14m 55s | 31m 04s | 1,1003 | 0,4802 |

Figure 10

| **Time (No compression)** | Total time Reads | total time unitigs | total time comp | speedup rate wrt reads | speedup rate wrt unitigs |
|---|---|---|---|---|---|
| Human Gen. | 1h 25m 9s | 22m 42s | 21m 57s | 3,8793 | 1,0342 |
| Human Gut Bact. | 6h 6m 31s | 50m 02s | 49m 22s | 7,4244 | 1,0135 |
| Salmonella | 1h 47m 37s | 38m 55s | 34m 12s | 3,1467 | 1,1379 |
| E. Coli | 2h 14m 49s | 59m 9s | 56m 44s | 2,3763 | 1,0426 |
| Drosophila | 34m 11s | 14m 55s | 14m 54s | 2,2942 | 1,0011 |

Figure 11

| Space | Total space reads GB | total space unitigs GB | Total space comp GB | Space needed wrt Reads | space needed wrt unitigs |
|---|---|---|---|---|---|
| Human Gen. | 33,017 | 7,41 | 4,021 | 12,18% | 54,26% |
| Human Gut Bact. | 99,275 | 20,963 | 10,813 | 10,89% | 51,58% |
| Salmonella | 35,791 | 7,312 | 3,75 | 10,48% | 51,29% |
| E. Coli | 116,811 | 25,293 | 12,533 | 10,73% | 49,55% |
| Drosophila | 30,717 | 5,588 | 2,926 | 9,53% | 52,36% |

### 10.3.2.2. Fulgor

Figure 12

| Time | Total time Reads | total time unitigs | total time comp | speedup rate wrt reads | speedup rate wrt unitigs |
|---|---|---|---|---|---|
| Human Gen. | 19m 10s | 3m 31s | 8m 20s | 2,3 | 0,422 |
| Human Gut Bact. | 1h 19m 38s | 12m 16s | 27m 2s | 2,9457 | 0,4538 |
| Salmonella | 25m 42s | 3m 40s | 7m 54s | 3,2532 | 0,4641 |
| E. Coli | 1h 24m 35s | 14m 3s | 16m 49s | 5,0297 | 0,8355 |
| Drosophila | 27m 51s | 3m 12s | 19m 18s | 1,443 | 0,1658 |

Figure 13

| Time (No compression) | Total time Reads | total time unitigs | total time comp | speedup rate wrt reads | speedup rate wrt unitigs |
|---|---|---|---|---|---|
| Human Gen. | 19m 10s | 3m 31s | 3m 25s | 5,6098 | 1,0293 |
| Human Gut Bact. | 1h 19m 38s | 12m 16s | 12m 16s | 6,4918 | 1 |
| Salmonella | 25m 42s | 3m 40s | 3m 21s | 7,6716 | 1,0945 |
| E. Coli | 1h 24m 35s | 14m 3s | 13m 10s | 6,4241 | 1,0671 |
| Drosophila | 27m 51s | 3m 12s | 3m 08s | 8,8883 | 1,0213 |

Figure 14

| Space | Total space reads GB | total space unitigs GB | Total space comp GB | Space needed wrt Reads | space needed wrt unitigs |
|---|---|---|---|---|---|
| Human Gen. | 37,872 | 7,084 | 3,695 | 9,76% | 52,16% |
| Human Gut Bact. | 117,68 | 20,548 | 10,398 | 8,84% | 50,60% |
| Salmonella | 44,226 | 7,273 | 3,701 | 8,37% | 50,89% |
| E. Coli | 138,742 | 25,956 | 12,836 | 9,25% | 49,45% |
| Drosophila | 37,007 | 5,539 | 2,877 | 7,77% | 51,94% |

### 10.3.2.3. GGCAT

Figure 15

| Time | Total time Reads | total time comp | speedup rate wrt reads |
|---|---|---|---|
| Human Gen. | 59m 03s | 19m 34s | 3,0179 |
| Human Gut Bact. | 4h 52m 6s | 45m 33s | 6,4127 |
| Salmonella | 1h 12m 9s | 38m 57s | 1,8524 |
| E. Coli | 49m 41s | 12m 40s | 3,9224 |
| Drosophila | 12m 24s | 18m 13s | 0,6807 |

Figure 16

| Time (No compression) | Total time Reads | total time comp | speedup rate wrt reads |
|---|---|---|---|
| Human Gen. | 59m 03s | 14m 39s | 4,0307 |
| Human Gut Bact. | 4h 52m 6s | 30m 47s | 9,4889 |
| Salmonella | 1h 12m 9s | 34m 24s | 2,0974 |
| E. Coli | 49m 41s | 8m 01s | 6,1975 |
| Drosophila | 12m 24s | 2m 03s | 6,0488 |

Figure 17

| Space | Total space reads GB | Total space comp GB | Space needed wrt Reads |
|---|---|---|---|
| Human Gen. | 38,697 | 6,759 | 17,47% |
| Human Gut Bact. | 126,032 | 17,378 | 13,79% |
| Salmonella | 42,432 | 7,078 | 16,68% |
| E. Coli | 144,716 | 21,101 | 14,58% |
| Drosophila | 37,342 | 5,396 | 14,45% |

### 10.3.2.4. Mash

Figure 18

| Time | Total time Reads | total time comp | speedup rate wrt reads |
|---|---|---|---|
| Human Gen. | 9m 17s | 5m 21s | 1,7352 |
| Human Gut Bact. | 41m 43s | 15m 48s | 2,6403 |
| Salmonella | 14m 5s | 5m 5s | 2,7705 |
| E. Coli | 38m 35s | 4m 6s | 9,4106 |
| Drosophila | 9m 40s | 16m 24s | 0,5894 |

Figure 19

| Time (No compression) | Total time Reads | total time comp | speedup rate wrt reads |
|---|---|---|---|
| Human Gen. | 9m 17s | 26s | 21,4231 |
| Human Gut Bact. | 41m 43s | 1m 2s | 40,371 |
| Salmonella | 14m 5s | 32s | 26,4063 |
| E. Coli | 38m 35s | 27s | 85,7407 |
| Drosophila | 9m 40s | 14s | 41,4286 |

Figure 20

| Space | Total space reads GB | Total space comp GB | Space needed wrt Reads |
|---|---|---|---|
| Human Gen. | 30,612 | 2,024 | 6,61% |
| Human Gut Bact. | 91,257 | 5,945 | 6,51% |
| Salmonella | 33,544 | 2,095 | 6,25% |
| E. Coli | 109,033 | 7,372 | 6,76% |
| Drosophila | 28,726 | 1,666 | 5,80% |